BiT

Bimodal Parabolic distribution based lattice signatures

Principal Submitter: Xianhui Lu[1,2], +86-15010131069, luxianhui@iie.ac.cn

Contributors:

- **Design:** Yongjian Yin[1,2], yinyongjian@iie.ac.cn

Yijian Liu[1,2], liuyijian@iie.ac.cn

Dongshu Cai[1,2], caidongshu@iie.ac.cn

- **Analysis:** Dingding Jia[1,2], jiadignding@iie.ac.cn

Lei Bi[1,2], bilei@iie.ac.cn

Yamin Liu[3], liuyamin3@huawei.com

Bing Shi[1,2], shibing@iie.ac.cn

Jingnan He[1,2], hejingnan@iie.ac.cn

- **Implementation:** Ya Shi[1,2], shiya@iie.ac.cn

Hang Zhang[1,2], zhanghang2024@iie.ac.cn

Ying Liu[1,2], liuying@iie.ac.cn

Yu Zhang[1,2], zhangyu@iie.ac.cn

Organizations: 1. State Key Laboratory of Cyberspace Security Defense,

Institute of Information Engineering, CAS

2. University of Chinese Academy of Sciences

3. Huawei Technologies

Owner: Xianhui Lu

Postal Address: No. 19, Shucun Road, Beijing 100093, China

Contact: liuying@iie.ac.cn, +86-18633352026

Contents

1 Introduction **5**

1.1 Innovation 5

1.2 Design Overview 5

1.3 Security and Results 9

2 Preliminaries **10**

2.1 Notations 10

2.2 Digital Signature Scheme 11

2.3 Lattice-Based Hard Problems 12

2.4 Rejection Sampling 13

2.5 Challenge Sampling 14

2.6 High/Low Bits Extraction 14

3 Algorithm Description **14**

3.1 Description of BiT 14

3.2 Size Reduction 15

3.3 Compressed Scheme Description 15

3.3.1 Rejection Conditions 16

3.4 Correctness 17

3.5 Rejection Counts 17

3.6 Pseudocode Description 18

3.7 Security for 512-bit 18

3.8 NTRU Version 20

4 Design Rationale **20**

4.1 Design Principles and Hard Problems 20

4.1.1 Design Principles 20

4.1.2 Mathematical Hard Problems 22

4.2 Criterion for Parameter Selection 22

4.3 The post-quantum security for the entropy of the challenge 22

4.3.1 Grover Search Algorithm 23

4.3.2 Quantum Circuit Depth 23

4.3.3 Quantum Security Corresponding to Different Challenge Entropies 24

5 Security Statements and Analysis **24**

5.1 Theoretical Security Analysis 24

5.2 Practical Security 29

5.3 Security Against Physical Attacks 30

6 Performance Evaluation **31**

6.1 Performance Analysis 31

6.2 Performance Test 32

6.2.1 x86 Platforms 32
6.2.2 Performance Test Results 32
6.3 Parameter, Security, and Size Summary 33

7 Feature Statements **33**

List of Figures

1 The Graph of Function f, F, G 8
2 BiT (framework) 9
3 The rejection-sampling algorithm RejectSample. 13
4 HighBits/LowBits 14
5 BiT 15
6 Compressed BiT 16
7 Pseudocode of BiT 19
8 BiT (NTRU Version) 20
9 Fiat-Shamir with Abort 21
10 Rejection Sampling in BiT 21
11 The Security Games 26

List of Tables

1 Public-key and signature sizes before and after the revision. 4
2 Efficiency, Security and Size 10
3 Efficiency, Security and Size (Short Size) 11
4 Function Declaration 18
5 Parameter Settings 23
6 MAXDEPTH 24
7 Quantum Security for Different Challenge Entropies 24
8 Practical Security (MLWE Version) 30
9 Efficiency and Size of SM3-Based Implementation 33
10 Efficiency of FIPS 202-Based Implementation 33
11 Parameter, Security, Size 35
12 Parameter, Security, Size for Compactness (Short Size) 36

Changes in This Revision

In the following, we list the changes between the originally submitted version of BiT and this revision, together with brief explanations of their motivation.

Changes to the Core BiT Design

- **Direct message binding for BiT-512.** In the submitted BiT-512 construction, the Fiat–Shamir challenge was computed from a fixed 512-bit message representative. Its generic collision resistance is therefore at most 256 bits, allowing a collision in the intermediate representative to transfer a signature from one message to another. In the revised BiT-512 construction, the message with delimited length is bound directly into the challenge:

$$\widetilde{c} = H_{\mathsf{chal}}(tr \parallel \mathrm{LE64}(|M|) \parallel M \parallel \mathrm{Encode}(w_1)).$$

  This removes the generic collision-security ceiling introduced by the intermediate fixed-length digest. If a standardized hash construction providing 512-bit classical collision security becomes available, BiT-512 can instead use the same fixed-length prehash structure as BiT-128 and BiT-256. Under the generic birthday bound, such a construction normally requires a message representative of at least 1024 bits.

- **Joint-vector parabolic rejection sampling.** The submitted construction applied rejection sampling coefficientwise while using one shared bimodal sign for the complete response. Although this corrected the individual coefficient marginals, it did not remove secret-dependent correlations between coordinates. The revised construction samples and rejects with respect to the joint distribution of the full response vector. For a secret-dependent shift $v = sc$ with $\|v\|_2 \le \beta$, the source distribution is

$$F_\Lambda(x, \gamma_1, v) = \tfrac{1}{2} f_\Lambda(x - v, \gamma_1) + \tfrac{1}{2} f_\Lambda(x + v, \gamma_1),$$

  and a candidate z is accepted with probability $\frac{G_\Lambda(z,\gamma_1,\beta)}{F_\Lambda(z,\gamma_1,v)}$. Here G_Λ is a parabolic target subdistribution defined over the joint coefficient embedding and depends only on public parameters. Thus the complete accepted response, rather than only each of its marginals, follows a secret-independent target distribution. This addresses the cross-coordinate leakage identified in `sign-02-2`.

- **Elimination of public-key compression.** The submitted construction transmitted only the high part b_1 of $b = A_0 s_0 + e$ and incorporated the omitted low part b_0 into hint generation. The revised public key is $pk = (\rho, b)$, where ρ expands to A_0 and the complete vector b is encoded without compression. Consequently, b_0 and all computations involving its hidden bimodal sign are removed from signing and verification. This eliminates the compressed-public-key leakage route associated with the analysis reported under `sign-07-3`. Signature compression by means of the hint h is unchanged.

Changes to Parameters, Implementations, and Documentation

- The parameter sets were retuned for the joint-vector rejection sampler and the uncompressed public-key representation. The concrete MLWE and MSIS estimates, including the classical Core-SVP estimates, were recomputed for the revised parameters.
- The reference and AVX2 implementations were updated to implement the revised challenge derivation, joint-vector rejection sampling, and full public-key encoding. The performance measurements and size tables were updated accordingly.
- The sizes of the revised primary parameter settings are summarized below. The submitted sizes are included only to make the effect of the design changes explicit.

Table 1: Public-key and signature sizes before and after the revision.

	Submitted version			Revised version		
Security level	$\|pk\|$	$\|\sigma\|$	Total	$\|pk\|$	$\|\sigma\|$	Total
128	1048	1504	2552	1264	1440	2704
256	2144	3456	5600	2528	2976	5504
512	5056	6695	11751	5824	7474	13298

1 Introduction

We propose BiT, a lattice-based digital signature scheme following the FiatShamir-with-aborts paradigm, whose security is based on the Module Short Integer Solution (MSIS) and Module Learning With Errors (MLWE) assumptions. We provide three parameter sets targeting three security levels: 128, 256, and 512 bits. We additionally describe a variant based on the NTRU cryptosystem.[1]

1.1 Innovation

This work improves rejection sampling for Fiat–Shamir lattice signatures by achieving stronger masking. Our rejection-sampling procedure adopts a bimodal structure of the form $\mathbf{y} + (-1)^b\mathbf{v}$, where the masking vector $\mathbf{y}$ is sampled from a parabolic distribution. The probability mass function of this distribution is proportional to $\gamma^2 - \|\mathbf{v}\|_2^2$ over its support. Here, γ represents the width of the masking distribution, and $||\mathbf{v}||_2$ is the 2-norm of the item to be masked.

More generally, the signing procedure first masks a secret-dependent term by masking distribution to obtain a source distribution. Rejection sampling then transforms this source distribution into a target distribution that is independent of the secret and can therefore be simulated using only public information. We optimize this transformation along two dimensions: the masking distribution[2] and the target distribution.

- **Masking distribution.** We use a parabolic distribution for masking. It can be viewed as a Gaussian-like distribution with sharply truncated tails, while retaining sampling complexity comparable to that of a Gaussian. More importantly, its shape provides stronger masking of the secret-dependent term, leading to a more favorable source distribution for rejection sampling.
- **Target distribution.** We also revisit the choice of the target distribution. In contrast to previous rejection-sampling approaches, which typically choose the target distribution from the same parameterization as the masking distribution, we decouple the two and use a parabolic target distribution with a different radius. This additional degree of freedom allows the rejection-sampling transformation to be optimized more effectively.

1.2 Design Overview

Our construction follows the Fiat-Shamir with Abort paradigm under the bimodal setting. We first review these two mechanisms separately.

(1) $\mathsf{Fiat-Shamir\ with\ Abort}$: Taking SIS-based schemes as an example, key generation follows the construction of SIS instances. The signing key is a small-norm matrix $\mathbf{S}$, and the verification key is $(\mathbf{A}, \mathbf{T} = \mathbf{AS} \bmod q)$, where $\mathbf{A}$ is uniformly sampled from $\mathbb{Z}_q^{n\times m}$. A signature on a message μ has the form $(\mathbf{z}, \mathbf{c})$, where $\mathbf{z} = \mathbf{y} + \mathbf{Sc}$, $\mathbf{y}$ is a masking vector used

[1] Compared with the MLWE-based BiT scheme, the NTRU-based variant offers no advantage in terms of size or performance. We therefore provide only its specification, without corresponding parameter sets.

[2] In the bimodal setting, optimizing the masking distribution amounts to optimizing the source distribution, since the former directly determines the latter.

to hide the secret term $\mathbf{Sc}$, and the challenge $\mathbf{c}$ is derived by feeding $\mathbf{Ay} \bmod q \parallel \mu$ into RO. Before outputting the signature, rejection sampling is performed to ensure that the signature distribution is independent of the signing key. Finally, the verification algorithm checks whether $\mathbf{z}$ is a short vector and verifies the equation $\mathbf{c} = \mathsf{H}(\mathbf{Az} - \mathbf{Tc} \bmod q, \mu)$, where H is RO.

(2) BimodalTechnique : In rejection sampling for $\mathbf{z} := \mathbf{y} + \mathbf{Sc}$, the term $\mathbf{v} := \mathbf{Sc}$ can be viewed as an offset applied to $\mathbf{y}$. Rejection sampling essentially removes this offset. Smaller offsets lead to simpler sampling and lower rejection rates.

The bimodal technique generates $\mathbf{z} = \mathbf{y} + (-1)^b\mathbf{Sc}$ with $b \leftarrow \{0, 1\}$. The two symmetric offsets $\pm\mathbf{Sc}$ partially cancel each other, effectively reducing the overall offset.

However, the additional randomness introduced by b breaks the standard verification equation. To complete the verification, it needs to modify the key generation such that $\mathbf{AS} = \hat{q}\mathbf{I}$, where $\hat{q} = 2^{-1} \bmod q$, and modifying the verification equation to $\mathbf{c} = \mathsf{H}(\mathsf{High}(\mathbf{Az} + \hat{q}\mathbf{c}), \mu)$, where High extracts the high-order bits of the input[3]. Under this setting, we obtain

$$\begin{aligned}\mathbf{Az} + \hat{q}\mathbf{c} &= \mathbf{A}[\mathbf{y} + (-1)^b\mathbf{Sc}] + \hat{q}\mathbf{c} \\ &= \mathbf{Ay} + [(-1)^b + 1]\hat{q}\mathbf{c} = \mathbf{Ay} + (1 - b)\mathbf{c}\end{aligned}$$

The second equation follows from $\mathbf{As} = \hat{q}\mathbf{I}_n$. Considering that when $b = 0$, $((-1)^b + 1)\hat{q} = 2\hat{q} = 1$, and when $b = 1$, $((-1)^b + 1)\hat{q} = 0$, we obtain the third equation. Since $\mathbf{c}$ is sufficiently small, $\mathsf{High}(\mathbf{Ay} + (1 - b)\mathbf{c})$ is nearly identical to $\mathsf{High}(\mathbf{Ay})$. Accordingly, the signing algorithm queries RO on $\mathsf{High}(\mathbf{Ay})||\mu$. Finally, the signing procedure adds the condition $\mathsf{High}(\mathbf{Ay} + (1 - b)\mathbf{c}) = \mathsf{High}(\mathbf{Ay})$ to ensure correct verification.

Rejection Sampling Optimization: We optimize the masking distribution and the target distribution, and the execution process.

(1) Masking Distribution-Sampling from Parabolic Distribution. Two masking distributions are commonly used in lattice-based signatures: Gaussian and uniform distributions. In the bimodal setting, Gaussian masking generally permits smaller masking parameters than uniform masking for a given rejection rate, and consequently leads to more compact signatures. For this reason, Gaussian distributions are widely used in state-of-the-art bimodal Fiat–Shamir lattice signatures.

We instead consider a parabolic masking distribution. Intuitively, compared with a Gaussian distribution of similar width, the parabolic distribution places more probability mass near the center while having bounded support. This shape is particularly well suited to bimodal rejection sampling: the secret-dependent source distributions have their largest common mass near the center, and the optimal target distribution is determined by their

[3]This technique originates from the NTRU+Sign signature scheme [15]. In the original bimodal scheme BLISS [8], the key relation is $\mathbf{AS} = q\mathbf{I}_n \bmod 2q$. We wish to retain $\bmod q$ instead of $\bmod 2q$.

common subdistribution. Concentrating the masking probability mass in this region therefore increases the exploitable overlap among the induced source distributions. This yields more effective masking than Gaussian sampling and allows the same target acceptance probability to be achieved with a substantially smaller masking radius, while retaining comparable sampling complexity.

Specifically, for the n-dimensional case, we define the parabolic distribution $f(\mathbf{x}, \gamma)$ by

$$f(\mathbf{x}, \gamma) = \begin{cases} C(\gamma^2 - \|\mathbf{x}\|_2^2), & \|\mathbf{x}\|_2 \leq \gamma, \\ 0, & \|\mathbf{x}\|_2 > \gamma, \end{cases}$$

where $C = \frac{n+2}{2V_n\gamma^{n+2}}$ and $V_n = \frac{\pi^{n/2}}{\Gamma(\frac{n}{2}+1)}$. Here, V_n is the volume of the n-dimensional unit ball. In particular, when n is even, $V_n = \frac{\pi^{n/2}}{(n/2)!}$.

(2) Target Distribution-Parabolic Distribution close to Parabolic Distribution.

In rejection sampling, the target distribution is introduced to compensate for the gap between the source distribution and the desired output distribution. Consequently, if the target distribution is chosen to be closer to the source distribution, the rejection rate can be reduced. In most previous schemes, the masking distribution itself is used as the target distribution. In contrast, our approach identifies the largest common region shared by all possible source distributions and then defines a new target distribution, with a different support, over this common region.

In our scheme, the source distribution is $\mathbf{z} := \mathbf{y} + (-1)^b\mathbf{v}$, where $\mathbf{y}$ is sampled from a parabolic masking distribution with radius γ, $\mathbf{v}$ is the secret-dependent term, and $b \leftarrow \{0, 1\}$ is the random bit introduced by the bimodal technique. The source distribution is then

$$F(\mathbf{x}, \gamma, \mathbf{v}) = \frac{f(\mathbf{x} - \mathbf{v}, \gamma)}{2} + \frac{f(\mathbf{x} + \mathbf{v}, \gamma)}{2}.$$

We construct the target distribution using only public information. Let β be a public bound such that $\|\mathbf{v}\|_2 \leq \beta$, where $0 < \beta < \gamma$. We define the target distribution by

$$G(\mathbf{x}, \gamma, \beta) = \begin{cases} \hat{C}(\gamma^2 - \beta^2 - \|\mathbf{x}\|_2^2), & \|\mathbf{x}\|_2 \leq \sqrt{\gamma^2 - \beta^2}, \\ 0, & \|\mathbf{x}\|_2 > \sqrt{\gamma^2 - \beta^2}. \end{cases}$$

Where

$$\hat{C} = \frac{n+2}{2V_n(\gamma^2 - \beta^2)^{(n+2)/2}}.$$

The masking distribution $f(\mathbf{x}, \gamma)$, the source distribution $F(\mathbf{x}, \gamma, \mathbf{v})$, and the target distribution $G(\mathbf{x}, \gamma, \beta)$ are illustrated in Figure 1, corresponding to the red, green, and blue curves, respectively. The green dashed curves represent $f(\mathbf{x} - \mathbf{v}, \gamma)$ and $f(\mathbf{x} + \mathbf{v}, \gamma)$,

and their superposition forms the distribution F. In this setting, the acceptance rate is

$$\rho = \left(1 - \frac{\beta^2}{\gamma^2}\right)^{\frac{n+2}{2}}.$$

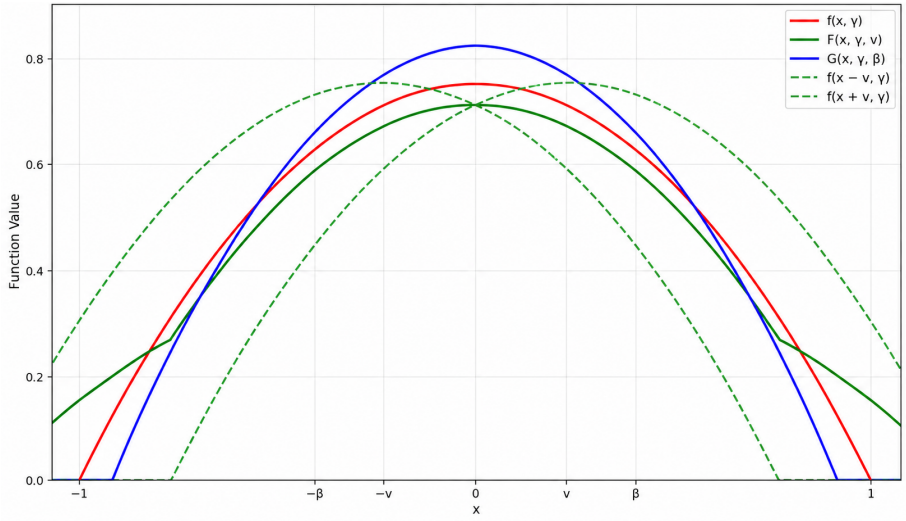


Figure 1: The Graph of Function f, F, G

Polynomial Ring: To further reduce the size of public keys and signatures, the proposed scheme is constructed over polynomial rings. The signing key $\mathbf{S}$ and verification key $\mathbf{T}$, which are matrices in the original paradigm, are replaced by polynomial vectors.

Within the bimodal framework, the signing key is defined as a short-norm polynomial vector $\mathbf{s} = (1, \mathbf{s}_0, \mathbf{e})$. The verification key is defined as a polynomial matrix $\mathbf{A} = [\hat{q}\mathbf{j}_n - \mathbf{b}, \mathbf{A}_0, \mathbf{I}]$, where $\mathbf{j}_n$ is a vector whose first entry is 1 and all other entries are 0, $\mathbf{A}_0$ is a uniformly random matrix, and $\mathbf{b} = \mathbf{A}_0\mathbf{s}_0 + \mathbf{e}$.

The public key does not need to store the entire matrix $\mathbf{A}$. Instead, it only stores the seed of $\mathbf{A}_0$ and the vector $\mathbf{b}$.

Modulus selection for arithmetic efficiency: We use an NTT-friendly prime q to enable efficient ring operations. The scheme performs all ring arithmetic using the Number Theoretic Transform (NTT) and uses a fully split polynomial ring to improve efficiency. For example, if the multiplicative group contains an element of order 256, a suitable prime such as $q = 7681$ allows the polynomial ring to split completely.

Overview of the Scheme: Based on the above design principles, the overall framework of the proposed scheme over $\mathbb{Z}_q$ is shown in Figure 2.

(1) KGen : The key generation algorithm outputs a short-norm secret key $\mathbf{S}$ and a public key $\mathbf{A}$ satisfying $\mathbf{AS} = \hat{q}\mathbf{I}_n$. Where $\hat{q} = 2^{-1} \bmod q$ and $\mathbf{I}_n$ is n-order identity matrix.

KGen(1^κ)

1 : $\mathbf{S} \in \mathbb{Z}_q^{m\times k}$
2 : $\mathbf{A} \in \mathbb{Z}_q^{n\times m}$
3 : $(\mathbf{A}, \mathbf{S})$ satisfy that $\mathbf{AS} = \hat{q}\mathbf{I}_n$
4 : $\mathsf{pk}, \mathsf{sk} = \mathbf{A}, \mathbf{S}$

Verify$(\sigma, \mathsf{pk}, \mu)$

1 : if $||\mathbf{z}||_2 > B_2$ or $||\mathbf{z}||_\infty > B_\infty$, return reject
2 : $\hat{\mathbf{w}}' = \mathbf{Az} + \hat{q}\mathbf{c}$
3 : $\mathbf{w}' = \mathsf{High}(\hat{\mathbf{w}}')$
4 : Accept if $\mathbf{c} = \mathsf{H}(\mathbf{w}', \mu)$

Sign$(\mathbf{A}, \mathsf{sk}, \mu)$

1 : $\mathbf{y} \in P_\gamma$
2 : $\mathbf{w} := \mathbf{Ay}$
3 : $\mathbf{c} = \mathsf{H}(\mathsf{High}(\mathbf{w}), \mu)$
4 : $b \xleftarrow{\$} \{0, 1\}$
5 : $\mathbf{z} := \mathbf{y} + (-1)^b\mathbf{Sc}$
6 : $\mathbf{z} := \mathsf{RejectSample}(\mathbf{z})$
7 : if $\mathbf{z} = \perp$, go into step 1
8 : if $\mathsf{High}(\mathbf{w}) \neq \mathsf{High}(\mathbf{w} + (1-b)\mathbf{c})$, go into step 1
9 : if $||\mathbf{z}||_2 > B_2$ or $||\mathbf{z}||_\infty > B_\infty$, go into step 1
10 : **return** $(\mathbf{z}, \mathbf{c})$

Figure 2: BiT (framework)

(2) Sign : To sign a message μ, the signing algorithm samples a masking vector $\mathbf{y}$ from the parabolic distribution P_γ. It then computes $\mathbf{Ay}$ and queries RO on $\mathsf{High}(\mathbf{Ay}) \parallel \mu$ to obtain the challenge $\mathbf{c}$. Here, $\mathbf{c}$ is an n-dimensional integer vector with exactly τ nonzero coefficients in $\{\pm 1\}$, and all remaining coefficients are 0. The function High extracts the high-order coefficients of $\mathbf{w}$. Next, the algorithm samples $b \leftarrow \{0, 1\}$ and computes $\mathbf{z} = \mathbf{y} + (-1)^b\mathbf{Sc}$.

To ensure correctness and security, several rejection checks are applied. Rejection sampling RejectSampling is used to truncate the signature distribution so that the output is independent of secret information. The condition $\mathsf{High}(\mathbf{w}) = \mathsf{High}(\mathbf{w} + (1-b)\mathbf{c})$ ensures correct verification while hiding the value of b, which encodes the offset direction of the secret term and must also be kept hidden from the adversary. The condition $\|\mathbf{z}\|_2 \leq B_2$ and $||\mathbf{z}||_\infty \leq B_\infty$ ensures a bounded signature norm, since otherwise an adversary can construct large signatures that still pass the remaining checks. Finally, the algorithm outputs the signature $(\mathbf{z}, \mathbf{c})$.

(3) Verify : To verify a signature, the verification algorithm first checks whether $||\mathbf{z}||_2 \leq B_2$ and $||\mathbf{z}||_\infty \leq B_\infty$. It then computes $\hat{\mathbf{w}}' = \mathbf{Az} + \hat{q}\mathbf{c}$ and $\mathbf{w}' = \mathsf{High}(\hat{\mathbf{w}}')$. Finally, the algorithm accepts the signature if $\mathbf{c} = \mathsf{H}(\mathbf{w}', \mu)$.

1.3 Security and Results

This section summarizes the efficiency, security strength and size of the scheme. Figure 2[4] provides a concise overview of the results. The security of the scheme is measured in terms of **core-SVP**. The values in parentheses correspond to the security levels for strong unforgeability of the randomized signature scheme. The complete parameters and test results are provided in Section 6.3.

[4]Performance results are based on the SM3-based implementation. Results for the FIPS 202-based implementation are presented in Section 6.2.2.

Table 2: Efficiency, Security and Size

Security Level	128	256	512
Efficiency AVX (clcyes)			
KGen	295,318	749,930	1,657,675
Sign	435,985	952,465	2,127,980
Verify	95,723	238,804	688,667
MLWE Hardness (Core-SVP)			
BKZ block-size	439	907	1816
Classical	128	264	531
Quantum	117	243	488
MSIS Hardness (Core-SVP)			
BKZ block-size (SUF)	560(442)	1183(880)	2379(1802)
Classical (SUF)	163(129)	345(256)	694(526)
Quantum (SUF)	148(117)	313(233)	630(477)
Size (bytes)			
pk	1264	2528	5824
sig	1440	2976	7474
pk+sig	2704	5504	13298

The efficiency of the BiT signature scheme in Table 2 is comparable to that of ML-DSA. In addition, we have provided the results of the short-sized version, as shown in Table 3.

2 Preliminaries

2.1 Notations

The operations mod and mod^+ are defined as follows: for $\forall a, q \in \mathbb{Z}$, it holds that $a \bmod q \in [-\frac{q}{2}, \frac{q}{2})$ and $a \bmod {}^+ q \in [0, q)$. For $n \in \mathbb{Z}$, denote $[n] = \{0, 1, \ldots, n-1\}$. $B(x, \gamma)$ and $\overline{B}(x, \gamma)$ denote the open and closed balls of radius γ centered at x, respectively.

Vectors are represented by bold lowercase letters, e.g. $\mathbf{v}$ and matrices by bold uppercase letters, e.g. $\mathbf{A}$. All vectors are assumed to be column vectors. The symbols $\mathbf{v}^T$ and $\mathbf{A}^T$ denote the transpose of the vector $\mathbf{v}$ and the matrix $\mathbf{A}$, respectively. Additionally, $\mathbf{v}_i$ indicates the i-th component of $\mathbf{v}$.

$\mathbb{R}_{>0}$ denotes $(0, +\infty) \cap \mathbb{R}$. $\mathbb{Z}_q$ and $\mathcal{R}_q$ denote $[-\frac{q}{2}, \frac{q}{2}) \cap \mathbb{Z}$ and $\mathbb{Z}_q[x]/\langle x^d + 1\rangle$, respectively. Let S_η denote the sparse ternary distribution over the ring $\mathcal{R}_q$, whose coefficients take values in $\{-1, 0, 1\}$. Here, η is a probability parameter: each of 1 and -1 is sampled with probability η, while 0 is sampled with probability $1 - 2\eta$. Let $\mathbf{j}_n = (1, 0, \ldots, 0)^T \in \mathcal{R}_q^n$. The coefficient vector of a polynomial $f(x) = a_0 + a_1 x + \cdots + a_{d-1} x^{d-1} \in \mathcal{R}_q$ is written as $(a_0, a_1, \ldots, a_{d-1})^T \in \mathbb{Z}_q^d$.

The ℓ_p-norm of a vector $\mathbf{v}$ is represented as $||\mathbf{v}||_p$. For $\mathbf{v} = (\mathbf{v}_0, \ldots, \mathbf{v}_{n-1})^T \in \mathbb{Z}_q^n$, the ℓ_p-norm is defined as $||\mathbf{v}||_p = (|\mathbf{v}_0|^p + \cdots + |\mathbf{v}_{n-1}|^p)^{1/p}$. For $\mathbf{w} = (\mathbf{w}_0, \ldots, \mathbf{w}_{n-1})^T \in \mathcal{R}_q^n$, $||\mathbf{w}||_p$ is defined as $[(||\mathbf{w}_0||_p)^p + \cdots + (||\mathbf{w}_{n-1}||_p)^p]^{1/p}$.

Table 3: Efficiency, Security and Size (Short Size)

Security Level	128	256	512
MLWE Hardness (Core-SVP)			
BKZ block-size	439	746	1335
Classical	128	263	512
Quantum	117	253	465
MSIS Hardness (Core-SVP)			
BKZ block-size (SUF)	516(452)	1097(894)	2192(1761)
Classical (SUF)	150(131)	320(261)	640(514)
Quantum (SUF)	136 (119)	290(236)	580(466)
pk (bytes)	1168	2336	5056
sig (bytes)	1104	2448	5520
pk+sig (bytes)	2272	4784	10576

For a distribution D, $x \xleftarrow{\$} D$ denotes sampling x from D. For a set S, $x \xleftarrow{\$} S$ denotes uniform sampling from S. Unless otherwise stated, all operations in $\mathbb{Z}_q$ and $\mathcal{R}_q$ are performed modulo q. All logarithms are base 2.

2.2 Digital Signature Scheme

Definition 1 (Digital Signature Scheme)**.** *A digital signature scheme consists of three probabilistic polynomial-time (PPT) algorithms:* $(\mathsf{KGen}, \mathsf{Sign}, \mathsf{Verify})$.

KGen*: Given* 1^κ *as input, generate the signing key* sk *and the verification key* pk*, where* κ *is the security parameter.*
Sign*: Given the signing key* sk *and a message* μ *as inputs, generate a signature* σ *for* μ *as output.*
Verify*: Given the verification key* pk*, a message* μ*, and a signature* σ *as inputs, output* 0 *to indicate rejection or* 1 *to indicate acceptance.*

Correctness. For all pairs $(\mathsf{pk}, \mathsf{sk}) \leftarrow \mathsf{KGen}(1^\kappa)$ and all messages μ, it always holds that $\mathsf{Verify}(\mathsf{pk}, \mu, \sigma) = 1$, where $\sigma \leftarrow \mathsf{Sign}(\mathsf{sk}, \mu)$.

Definition 2 (Unforgeability under Chosen Message Attack)**.** *A digital signature scheme is called* (t, ϵ, q_S, q_H)-$\mathsf{UF\text{-}CMA}$ *secure (unforgeability under chosen message attack) if, for any PPT adversary* $\mathcal{A}$ *with running time* t*,* q_S *signature queries and* q_H *hash queries, there is:*

$$\Pr\left[\mathsf{Verify}(\mathsf{pk}, \mu, \sigma) = 1 \wedge \mu \notin M_{Sign} : (\mu, \sigma) \leftarrow \mathcal{A}^{\mathcal{O}_S, \mathcal{O}_H}(\mathsf{pk})\right] \leq \epsilon,$$

where $\mathcal{O}_S$ *and* $\mathcal{O}_H$ *denote the signing and hash oracles, which respond with signatures and hash values, respectively.* M_{Sign} *is the set of messages that have been queried to* $\mathcal{O}_S$*.*

Based on the definition of $\mathsf{UF\text{-}CMA}$ security, other security notions can be classified as follows. If $q_S = 0$, it is referred to as (t, ϵ, q_H)-$\mathsf{UF\text{-}NMA}$-secure (unforgeability under no message attack);

If the adversary is allowed to forge a new signature for a message that has already been queried, the security notion becomes (t, ϵ, q_S, q_H)-SUF-CMA-secure (strong unforgeability under chosen message attack).

2.3 Lattice-Based Hard Problems

Definition 3 (Search $\mathsf{MSIS}_{q,n,m,\beta}$ Problem)**.** *The search $\mathsf{MSIS}_{q,n,m,\beta}$ problem is called (t, ϵ)-hard if, for any PPT adversary $\mathcal{A}$ with running time t,*

$$Adv_{\mathcal{A}}^{\mathsf{MSIS}} = \Pr[[\mathbf{I}|\mathbf{A}] \cdot \mathbf{s} = \mathbf{0} \wedge ||\mathbf{s}||_2 \leq \beta | \mathbf{A} \xleftarrow{\$} \mathcal{R}_q^{n \times m}; \mathbf{s} \in \mathcal{R}_q^{n+m} / \{\mathbf{0}\} \leftarrow \mathcal{A}(\mathbf{A})] \leq \epsilon$$

Definition 4 ($\mathsf{MLWE}_{q,n,m,\phi,\varphi}$ instance)**.** *For parameters q, n, m, two probability distributions ϕ and φ on $\mathcal{R}_q$, a $\mathsf{MLWE}_{q,n,m,\phi,\varphi}$ instance is that, randomly selecting a matrix $\mathbf{A} \xleftarrow{\$} \mathcal{R}_q^{n \times m}$, a pair $(\mathbf{s}, \mathbf{e}) \in \phi^m \times \varphi^n$, and outputting $(\mathbf{A}, \mathbf{t} = \mathbf{A}\mathbf{s} + \mathbf{e} \bmod q)$.*

Definition 5 (Decision $\mathsf{MLWE}_{q,n,m,\phi,\varphi}$ Problem)**.** *The decision $\mathsf{MLWE}_{q,n,m,\phi,\varphi}$ problem is called (t, ϵ)-hard if, for any PPT adversary $\mathcal{A}$ with running time t,*

$$Adv_{\mathcal{A}}^{\mathsf{MLWE}} = |\Pr[1 \leftarrow \mathcal{A}^{\mathcal{O}_{\mathsf{MLWE}}}(1^\kappa)] - \Pr[1 \leftarrow \mathcal{A}^{\mathcal{O}_{\mathsf{uniform}}}(1^\kappa)]| \leq \epsilon$$

where $\mathcal{O}_{\mathsf{MLWE}}$ and $\mathcal{O}_{\mathsf{uniform}}$ denote the MLWE and uniform oracles, respectively. The oracle $\mathcal{O}_{\mathsf{MLWE}}$ outputs MLWE instances, while $\mathcal{O}_{\mathsf{uniform}}$ returns a uniformly random pair $(\mathbf{A}, \mathbf{t}) \xleftarrow{\$} \mathcal{R}_q^{n \times m} \times \mathcal{R}_q^n$.

Definition 6 (The $\mathsf{BimodalSelfTargetMSIS}$ problem)**.** *For parameters $q, n, m, \gamma \in \mathbb{N}$, the ring $\mathcal{R}_q$ and the hash function $\mathsf{H} : \{0,1\}^* \to \mathcal{C} \subset \mathcal{R}_q$, the $\mathsf{BimodalSelfTargetMSIS}$ problem is called (t, ϵ)-hard, if for any PPT adversary $\mathcal{A}$ with running time t,*

$$\Pr\left[\begin{matrix} 0 \leq ||\mathbf{y}||_2 \leq \gamma \\ \mathsf{H}(\mathbf{A}\mathbf{y} + \hat{q}\mathbf{j}_n c, \mu) = c \end{matrix} \middle| \begin{matrix} (\mathbf{A}_0, \mathbf{b}) \xleftarrow{\$} \mathcal{R}_q^{n \times m} \times \mathcal{R}_q^n; (\mathbf{y}, \mu, c) \leftarrow \mathcal{A}^{\mathcal{O}_H}(\mathbf{A}) \\ \mathbf{A} := [\hat{q}\mathbf{j}_n - \mathbf{b}, \mathbf{A}_0, \mathbf{I}_n] \end{matrix}\right] \leq \epsilon,$$

where $\mathbf{y} \in \mathcal{R}_q^{n+m+1}$, μ is a string and $c \in \mathcal{R}_q$, and $\mathcal{O}_H$ denotes the hash oracle. The oracle $\mathcal{O}_H$ outputs hash values for queries.

Definition 7 ($\mathsf{NTRU}_{q,\phi}$ instance)**.** *For parameters q, $\hat{q} = 2^{-1} \bmod q$, a probability distribution ϕ on $\mathcal{R}_q$, an $\mathsf{NTRU}_{q,\phi}$ instance is that, randomly selecting two polynomial $\mathbf{f}, \mathbf{g} \xleftarrow{\$} \phi^2$, where $\mathbf{f}$ is invertible, and outputting $(\mathbf{g} + \hat{q})/\mathbf{f}$.*

Definition 8 (Decisional $\mathsf{NTRU}_{q,\phi}$ problem)**.** *The decisional $\mathsf{NTRU}_{q,\phi}$ problem is (t, ϵ)-hard if, for any PPT adversary $\mathcal{A}$ with running time t,*

$$Adv_{\mathcal{A}}^{\mathsf{NTRU}} = |\Pr[1 \leftarrow \mathcal{A}^{\mathcal{O}_{\mathsf{NTRU}}}(1^\kappa)] - \Pr[1 \leftarrow \mathcal{A}^{\mathcal{O}_{\mathsf{uniform}}}(1^\kappa)]| \leq \epsilon,$$

where the oracle $\mathcal{O}_{\mathsf{NTRU}}$ replies with NTRU instances, and the oracle $\mathcal{O}_{\mathsf{uniform}}$ replies with the uniform samples from $\mathcal{R}_q$. κ is the security parameter.

2.4 Rejection Sampling

Let $\mathcal{R}_q := \mathbb{Z}_q[x]/\langle x^d+1\rangle$, and let Λ denote the lattice induced by the coefficient embedding of $\mathcal{R}_q^m$ into an md-dimensional Euclidean space. We define the discrete parabolic masking distribution over Λ by

$$f_\Lambda(\mathbf{x},\gamma) = \begin{cases} \dfrac{\gamma^2 - \|\mathbf{x}\|_2^2}{Z_{\Lambda,\gamma}}, & \|\mathbf{x}\|_2 \le \gamma \ \wedge \ \mathbf{x} \in \Lambda, \\ 0, & \text{otherwise}, \end{cases} \tag{1}$$

where $Z_{\Lambda,\gamma} := \sum_{\substack{\mathbf{x}\in\Lambda \\ \|\mathbf{x}\|_2\le\gamma}} (\gamma^2 - \|\mathbf{x}\|_2^2)$ is the normalization factor.

For the signature component $\mathbf{z} := \mathbf{y}+(-1)^b\mathbf{v}$, where $b \xleftarrow{\$} \{0,1\}$ and $\mathbf{v}$ is the secret-dependent term, the induced source distribution is

$$F_\Lambda(\mathbf{x},\gamma,\mathbf{v}) = \frac{1}{2}f_\Lambda(\mathbf{x}-\mathbf{v},\gamma) + \frac{1}{2}f_\Lambda(\mathbf{x}+\mathbf{v},\gamma).$$

Then, we define the following target subdistribution:

$$G_\Lambda(\mathbf{x},\gamma,\beta) = \begin{cases} \dfrac{\gamma^2 - \beta^2 - \|\mathbf{x}\|_2^2}{Z_{\Lambda,\gamma}}, & \|\mathbf{x}\|_2 \le \sqrt{\gamma^2-\beta^2} \ \wedge \ \mathbf{x} \in \Lambda, \\ 0, & \text{otherwise}, \end{cases}$$

where β is a public upper bound on the norm of the secret-dependent shift $\mathbf{v}$. Unlike f_Λ, the function G_Λ is intentionally not normalized to have total mass one. Its total mass

$$\rho := \sum_{\mathbf{x}\in\Lambda} G_\Lambda(\mathbf{x},\gamma,\beta) = \frac{Z_{\Lambda,\sqrt{\gamma^2-\beta^2}}}{Z_{\Lambda,\gamma}}, \tag{2}$$

where $Z_{\Lambda,\sqrt{\gamma^2-\beta^2}} := \sum_{\mathbf{x}\in\Lambda\wedge\|\mathbf{x}\|_2\le\sqrt{\gamma^2-\beta^2}} (\gamma^2-\beta^2-\|\mathbf{x}\|_2^2)$, is exactly the acceptance probability of the rejection-sampling procedure. Consequently, the expected number of iterations is $1/\rho$.

Given a candidate signature component $\mathbf{z}$, the rejection-sampling algorithm RejectSample accepts $\mathbf{z}$ with probability $\frac{G_\Lambda(\mathbf{z},\gamma,\beta)}{F_\Lambda(\mathbf{z},\gamma,\mathbf{v})}$. The resulting rejection-sampling procedure is given in Figure 3.

RejectSample$(\mathbf{z},\gamma,\mathbf{v},\beta)$

1 : $F \leftarrow F_\Lambda(\mathbf{z},\gamma,\mathbf{v})$
2 : $G \leftarrow G_\Lambda(\mathbf{z},\gamma,\beta)$
3 : return $\mathbf{z}$ with probability G/F
4 : otherwise return $\perp$

Figure 3: The rejection-sampling algorithm RejectSample.

2.5 Challenge Sampling

The adapted hash function maps the input to the ball B_τ. Here, B_τ is defined as a set of polynomials over $\mathcal{R}_q$, where each polynomial contains exactly τ coefficients of 1 and zeros for the remaining coefficients, with its size given by $|B_\tau| = \binom{d}{\tau}$.

The hash implementation in the proposed signature scheme consists of two steps. First, a hash function maps $\{0,1\}^*$ to $\{0,1\}^d$. This step enables a concise and compact representation of elements in B_τ, and its output is much shorter than that of the second step. Incorporating the hash value generated in the first step into the signature effectively reduces the overall signature size. In the second step, an XOF is utilized to map the output of the first stage to an element in B_τ for subsequent computation. The detailed description of the second-step function is presented as follows:

SampleInBall(ρ)
01 initialize $\mathbf{c} = c_0 c_1 \dots c_{d-1} = 00 \dots 0$
02 for $i := d - \tau$ to $d - 1$
03 $\quad j \xleftarrow{\$} \{0, 1, \dots, i\}$
04 $\quad c_i := c_j$
05 $\quad c_j := 1$
06 return $\mathbf{c}$

2.6 High/Low Bits Extraction

The function HighBits and LowBits extract the high and low bits of the input through modulo operation, as detailed in Figure 4.

LowBits($\mathbf{w}, \gamma$)	**HighBits**($\mathbf{w}, \gamma$)
1 : return $\mathbf{w} \bmod \gamma$	1 : return $[\mathbf{w} - \mathsf{LowBits}(\mathbf{w})]/\gamma$

Figure 4: HighBits/LowBits

3 Algorithm Description

3.1 Description of BiT

Following the framework in Figure 2, we formally present the MLWE variant of the BiT signature scheme. The underlying lattice is shifted from the integer lattice $\mathbb{Z}_q$ to the polynomial ring $\mathcal{R}_q$. The full description of BiT is given in Figure 5. Furthermore, we rewrite the f_{Λ,γ_1} defined in Equation 1 as P_{γ_1} to represent the parabolic distribution.

Since the construction is defined over $\mathcal{R}_q$, the condition $\mathbf{AS} = \hat{q}\mathbf{I}_n$ is replaced by $\mathbf{As} = \hat{q}\mathbf{j}_n$, with corresponding modifications throughout the scheme. Here, $\mathbf{j}_n = (1, 0, \dots, 0)^T \in \mathcal{R}_q^n$, $\mathbf{A}$ is a polynomial matrix, and $\mathbf{s}$ is a polynomial vector. We present the concrete construction of KGen, whose security relies on the hardness of the MLWE problem.

KGen(1^κ)

1 : $\mathbf{s}_0, \mathbf{e} \xleftarrow{\$} S_\eta^m \times S_\eta^n \subset \mathcal{R}_q^m \times \mathcal{R}_q^n$
2 : $\mathbf{A}_0 \xleftarrow{\$} \mathcal{R}_q^{n \times m}$
3 : $\mathbf{b} = \mathbf{A}_0\mathbf{s}_0 + \mathbf{e}$
4 : $\mathbf{A} = [\hat{q}\mathbf{j}_n - \mathbf{b}, \mathbf{A}_0, \mathbf{I}_n]$
5 : $\mathbf{s} := (1, \mathbf{s}_0^T, \mathbf{e}^T)^T$
6 : if $\mathcal{N}(\mathbf{s}) > d\beta^2/\tau$, go into step 1
7 : $\mathsf{pk} := \mathbf{A}; \mathsf{sk} := \mathbf{s}$

Verify$(\mathsf{pk}, \sigma, \mu)$

1 : if $||\mathbf{z}||_2 > B_2$ or $||\mathbf{z}||_\infty > B_\infty$, then reject
2 : $\hat{\mathbf{w}}' = \mathbf{A}\mathbf{z} + \hat{q}\mathbf{j}_n c$
3 : $\mathbf{w}' = \mathsf{HighBits}(\hat{\mathbf{w}}', \gamma_2)$
4 : Accept if $c = \mathsf{H}(\mathbf{w}', \mu)$

Sign$(\mathsf{pk}, \mathsf{sk}, \mu)$

1 : $\mathbf{y} \xleftarrow{\$} P_{\gamma_1}$
2 : $\mathbf{w} := \mathbf{A}\mathbf{y}$
3 : $c \in \mathcal{R}_q := \mathsf{H}(\mathsf{HighBits}(\mathbf{w}, \gamma_2), \mu)$
4 : $b \xleftarrow{\$} \{0, 1\}$
5 : $\mathbf{z} := \mathbf{y} + (-1)^b \mathbf{s}c$
6 : $\mathbf{z} := \mathsf{RejectSample}(\mathbf{z}, \gamma_1, \mathbf{s}c, \beta)$
7 : if $\mathbf{z} = \perp$, go into step 1
8 : if $\mathsf{HighBits}(\mathbf{w}, \gamma_2) \neq \mathsf{HighBits}(\mathbf{w} + (-1)^b \mathbf{j}_n c, \gamma_2)$, go into step 1
9 : if $||\mathbf{z}||_2 > B_2$ or $||\mathbf{z}||_\infty > B_\infty$, go into step 1
10 : **return** $\sigma := (\mathbf{z}, c)$

Figure 5: BiT

3.2 Size Reduction

Combined with the special structure of the signing and public keys in the MLWE setting, the signature size can be further reduced.

Rejection sampling on the secret key We follow the rejection sampling on secret key in HAETAE [7] to make the signature shorter and the forgery harder. We define $\mathcal{N}(s) := \tau \cdot \sum_{i=1}^{\lfloor d/\tau \rfloor} \max_{0 \le j < 2d}^{i\text{-th}} \|s(\zeta^j)\|_2^2 + r \cdot \max_{0 \le j < 2d}^{(\lfloor d/\tau \rfloor + 1)\text{-th}} \|s(\zeta^j)\|_2^2 \le \gamma^2 n.$, where i-th over max means the i-th largest value and ζ is a $2d$-th root of unit. Therefore, given the bound $\mathcal{N}(\mathbf{s}) > 2d(\beta^2/\tau)$, we have $\forall c, \|c\mathbf{s}\|_2^2 \le \beta^2$.

Signature Compression by hint In the BiT signature scheme, $\mathbf{A} = [\hat{q}\mathbf{j}_n - \mathbf{b}, \mathbf{A}_0, \mathbf{I}_n]$ can be rewritten as $[\mathbf{A}_1, \mathbf{I}_n]$, where $\mathbf{A}_1 = [\hat{q}\mathbf{j}_n - \mathbf{b}, \mathbf{A}_0]$. Similarly, the signing key $\mathbf{s} := (1, \mathbf{s}_0^T, \mathbf{e}^T)^T$ can be decomposed as $(\mathbf{s}_1^T, \mathbf{s}_2^T)^T$, where $\mathbf{s}_1 = (1, \mathbf{s}_0^T)$ and $\mathbf{s}_2 = \mathbf{e}$.

Accordingly, the masking vector is split into $(\mathbf{y}_1, \mathbf{y}_2)$[5] to mask $\mathbf{s}_1$ and $\mathbf{s}_2$, respectively. As a result, the signature format changes from $(\mathbf{z}, c)$ to $(\mathbf{z}_1, \mathbf{z}_2, c)$.

During verification, the core condition $c = \mathsf{H}(\mathsf{HighBits}(\mathbf{A}\mathbf{z} + \hat{q}\mathbf{j}_n c), \mu)$ is replaced with $c = \mathsf{H}(\mathsf{HighBits}(\mathbf{A}_1\mathbf{z}_1 + \mathbf{z}_2 + \hat{q}\mathbf{j}_n c), \mu)$. Since $\mathbf{z}_2$ is short, $\mathsf{HighBits}(\mathbf{A}_1\mathbf{z}_1 + \mathbf{z}_2 + \hat{q}\mathbf{j}_n c)$ is almost identical to $\mathsf{HighBits}(\mathbf{A}_1\mathbf{z}_1 + \hat{q}\mathbf{j}_n c)$ with differences arising only from carry and borrow errors introduced by $\mathbf{z}_2$. Therefore, we replace the full vector $\mathbf{z}_2$ with a compact hint $\mathbf{h}$, reducing the signature size by nearly half and outputting only $(\mathbf{z}_1, \mathbf{h}, c)$.

3.3 Compressed Scheme Description

The description of the Compressed BiT signature scheme is shown in Figure 6. We illustrate the modifications to the scheme in Figure 6 compared with Figure 5.

[5]The input to RO is changed to $\mathsf{HighBits}(\mathbf{A}_1\mathbf{y}_1 + \mathbf{y}_2, \gamma_2) \parallel \mu$.

KGen(1^κ)

1 : $(\mathbf{s}_0, \mathbf{e}) \xleftarrow{\$} S_\eta^m \times S_\eta^n \subset \mathcal{R}_q^m \times \mathcal{R}_q^n$
2 : $\mathbf{A}_0 \xleftarrow{\$} \mathcal{R}_q^{n \times m}$
3 : $\mathbf{b} := \mathbf{A}_0\mathbf{s}_0 + \mathbf{e}$
4 : $\mathbf{A} = [\hat{q}\mathbf{j}_n - \mathbf{b}, \mathbf{A}_0, \mathbf{I}_n] = [\mathbf{A}_{1,s_0}, \mathbf{I}_n]$
5 : $\mathbf{s} := (1, \mathbf{s}_0^T, \mathbf{e}^T)^T$
6 : if $\mathcal{N}(\mathbf{s}) \leq d\beta^2/\tau$ go into step 1
7 : $\mathsf{pk} := \mathbf{A}; \mathsf{sk} := \mathbf{s}$

Verify$(\mathsf{pk}, \sigma, \mu)$

1 : if $||(\mathbf{z}_{1,s_0}, \gamma_2\mathbf{h})||_2 > B_2$ or $||(\mathbf{z}_{1,s_0}, \gamma_2\mathbf{h})||_\infty > B_\infty$, then reject
2 : $\hat{\mathbf{w}}' = \mathbf{A}_{1,s_0}\mathbf{z}_{1,s_0} + \hat{q}\mathbf{j}_n c$
3 : $\mathbf{w}' = \mathsf{HighBits}(\hat{\mathbf{w}}', \gamma_2) + \mathbf{h}$
4 : Accept if $c = \mathsf{H}(\mathbf{w}', \mu)$

Sign$(\mathsf{pk}, \mathsf{sk}, \mu)$

1 : $\mathbf{y} \xleftarrow{\$} P_{\gamma_1}$
2 : $\mathbf{w} := \mathbf{A}\mathbf{y}$
3 : $c \in \mathcal{R}_q := \mathsf{H}(\mathsf{HighBits}(\mathbf{w}, \gamma_2), \mu)$
4 : $b \xleftarrow{\$} \{0, 1\}$
5 : $\mathbf{z} := \mathbf{y} + (-1)^b\mathbf{s}c$
6 : $\mathbf{z} := \mathsf{RejectSample}(\mathbf{z}, \gamma_1, \mathbf{s}c, \beta)$
7 : $(\mathbf{z}_{1,s_0}, \mathbf{z}_e) \in \mathcal{R}_q^{m+1} \times \mathcal{R}_q^n := \mathsf{Split}(\mathbf{z})$
8 : if $\mathsf{HighBits}(\mathbf{w}, \gamma_2) \neq \mathsf{HighBits}(\mathbf{w} + (-1)^b\mathbf{j}_n c, \gamma_2)$, go into step 1
9 : $\hat{\mathbf{w}} = \mathbf{w} - \mathbf{z}_e + (1-b)\mathbf{j}_n c$
10 : $\mathbf{h} = \mathsf{HighBits}(\mathbf{w}, \gamma_2) - \mathsf{HighBits}(\hat{\mathbf{w}}, \gamma_2)$
11 : if $||(\mathbf{z}_{1,s_0}, \gamma_2\mathbf{h})||_2 > B_2$ or $||(\mathbf{z}_{1,s_0}, \gamma_2\mathbf{h})||_\infty > B_\infty$, then restart
12 : **return** $\sigma := (\mathbf{z}_{1,s_0}, \mathbf{h}, c)$

Figure 6: Compressed BiT

In Sign, following the signature compression in Section 3.2, the algorithm no longer outputs the component $\mathbf{z}_e$ of the signature $\mathbf{z}$ that masks $\mathbf{e}$. Sign generates the hint $\mathbf{h}$ for $\mathbf{z}_e$ to compress the signature (Steps 9–10). In addition, the norm verification of $\mathbf{z} := (\mathbf{z}_{1,s_0}, \mathbf{z}_e)$ is replaced with that of $(\mathbf{z}_{1,s_0}, \gamma_2\mathbf{h})$ (Step 11). Considering that $\mathbf{h}$ can be regarded as $\mathbf{z}_e$ mod γ_2, this adjustment is effectively equivalent to the original norm check on $\mathbf{z}$.

In Verify, the algorithm uses the hint $\mathbf{h}$ to complete the signature verification (Step 3).

3.3.1 Rejection Conditions

There are three rejection conditions in Sign: (1) $\mathbf{z} := \mathsf{RejectSample}(\mathbf{z})$ (Steps 6); (2) $\mathsf{HighBits}(\mathbf{w}, \gamma_2) \neq \mathsf{HighBits}(\mathbf{w} + (-1)^b\mathbf{j}_n c, \gamma_2)$ (Step 8); (3) $||(\mathbf{z}_{1,s_0}, \gamma_2\mathbf{h})||_2 > B_2$ and $||(\mathbf{z}_{1,s_0}, \gamma_2\mathbf{h})||_\infty > B_\infty$ (Step 11). Their functions are explained here.

(1) $\mathbf{z} := \mathsf{RejectSample}(\mathbf{z})$: This condition ensures that $\mathbf{z}_{1,s_0}$ and $\mathbf{h}$ in the signature do not reveal information about the key $\mathbf{s}$, where $\mathbf{h}$ can be seen as a compression of $\mathbf{z}_2$.

(2) $\mathsf{HighBits}(\mathbf{w}, \gamma_2) \neq \mathsf{HighBits}(\mathbf{w} + (-1)^b\mathbf{j}_n c, \gamma_2)$: Without the signature compression techniques of Section 3.2, this check ensures normal signature verification, as verifiers cannot directly access b. In the modified scheme, where b is embedded in the hint $\mathbf{h}$, the check prevents adversaries from gaining information about b.

Specifically, b can take two values, 0 or 1, which lead to different distributions of $\mathbf{h}$. When $b = 0$, $\mathbf{h} = \mathsf{HighBits}(\mathbf{w}, \gamma_2) - \mathsf{HighBits}(\mathbf{w} - \mathbf{z}_e + \mathbf{j}_n c, \gamma_2)$, and when $b = 1$, $\mathbf{h} = \mathsf{HighBits}(\mathbf{w}, \gamma_2) - \mathsf{HighBits}(\mathbf{w} - \mathbf{z}_e, \gamma_2)$. This check ensures that $\mathsf{HighBits}(\mathbf{w}, \gamma_2) = \mathsf{HighBits}(\mathbf{w} + (-1)^b\mathbf{j}_n c, \gamma_2)$, so that for $b = 0$, $\mathbf{h} = \mathsf{HighBits}(\mathbf{w}, \gamma_2) - \mathsf{HighBits}(\mathbf{w} - \mathbf{z}_e + \mathbf{j}_n c, \gamma_2) = \mathsf{HighBits}(\mathbf{w}, \gamma_2) - \mathsf{HighBits}(\mathbf{w} - \mathbf{z}_e, \gamma_2)$, matching the case when $b = 1$. This ensures consistency in $\mathbf{h}$ and prevents leakage of b.

(3) $||(\mathbf{z}_{1,s_0}, \gamma_2\mathbf{h})||_2 > B_2$ and $||(\mathbf{z}_{1,s_0}, \gamma_2\mathbf{h})||_\infty > B_\infty$: This step checks the norm of $(\mathbf{z}_{1,s_0}, \gamma_2\mathbf{h})$ to ensure that the signature remains within the prescribed size bound. This prevents an adversary from producing an overly large signature that nevertheless satisfies the remaining

verification conditions. The bound B_2 is defined as $B_2 = \sqrt{\frac{\gamma_1^2(m+1)}{m+n+1} + \gamma_2^2 B_{h_2}}$. Since the norm check is applied to $(\mathbf{z}_{1,s_0}, \gamma_2\mathbf{h})$, the signature explicitly contains only the first $m+1$ coordinates of $\mathbf{z}$, while the remaining n coordinates are represented compactly by the hint $\mathbf{h}$, whose ℓ_2-norm is bounded by B_{h_2}.

3.4 Correctness

Theorem 1. *The signature scheme described in Figure 6 is perfectly correct.*

Proof. According to the text $||(\mathbf{z}_{1,s_0}, \gamma_2\mathbf{h})||_2 > B_2$ and $||(\mathbf{z}_{1,s_0}, \gamma_2\mathbf{h})||_\infty > B_\infty$ in the step 11 of Sign, for the valid signature, $||(\mathbf{z}_{1,s_0}, \gamma_2\mathbf{h})||_\infty \leq B_2$ and $||(\mathbf{z}_{1,s_0}, \gamma_2\mathbf{h})||_\infty \leq B_\infty$ always hold true. Thus, we only fucus on the condition $\mathbf{c} = \mathsf{H}(\mathbf{w}', \mu)$ in Verify. Let $\mathbf{y} := (\mathbf{y}_{1,s_0}, \mathbf{y}_e) \in \mathcal{R}_q^{m+1} \times \mathcal{R}_q^n$.

$$
\begin{aligned}
\hat{\mathbf{w}}' &= \mathbf{A}_{1,s_0}\mathbf{z}_{1,s_0} + \hat{q}\mathbf{j}_n c \\
&= \mathbf{A}_{1,s_0}[\mathbf{y}_{1,s_0} + (-1)^b \mathbf{s}_{1,s_0} c] + \hat{q}\mathbf{j}_n c + [\mathbf{y}_e + (-1)^b \mathbf{s}_{1,s_0} c] - \mathbf{z}_e \\
&= (\mathbf{A}_{1,s_0}\mathbf{y}_{1,s_0} + \mathbf{y}_e) + (-1)^b(\mathbf{A}_{1,s_0}\mathbf{s}_{1,s_0} + \mathbf{e})c + \hat{q}\mathbf{j}_n c - \mathbf{z}_e \\
&= \mathbf{A}\mathbf{y} + [(-1)^b + 1]\hat{q}\mathbf{j}_n c - \mathbf{z}_e \\
&= \mathbf{A}\mathbf{y} - \mathbf{z}_e + (1-b)\mathbf{j}_n c = \hat{\mathbf{w}}
\end{aligned}
$$

The fourth equation comes from $\mathbf{A}\mathbf{s} = \hat{q}\mathbf{j}_n$. Considering that when $b = 0$, $((-1)^b + 1)\hat{q} = 2\hat{q} = 1$, and when $b = 1$, $((-1)^b + 1)\hat{q} = 0$, we obtain the fifth equation. Thus, $\mathbf{w}' = \mathsf{HighBits}(\mathbf{w}, \gamma_2)$. This means that $\mathbf{c} = \mathsf{H}(\mathbf{w}', \mu)$. □

3.5 Rejection Counts

Lemma 1. *In the scheme in Figure 6, the heuristic expected repetition time is less than $M = 1/\rho$, where $\rho \approx \left(1 - \beta^2/\gamma^2\right)^{\frac{(m+n+1)d+2}{2}} \cdot (1 - 1/\gamma_2)^{n\tau} > 0$ represents the probability of generating a valid signature within a single rewind, where d is the degree of the polynomial.*

Proof. According to the analysis in Section 3.3, rejections in Sign arise from two sources: (1) $\mathbf{z} := \mathsf{RejectSample}(\mathbf{z})$ and (2) $\mathsf{HighBits}(\mathbf{w}, \gamma_2) \neq \mathsf{HighBits}(\mathbf{w} + (-1)^b\mathbf{j}_n c, \gamma_2)$, with the first being the primary cause. As analyzed in Section 2.4, its acceptance rate is the result of the equation 2. Thus,

$$
\rho := \frac{Z_{\Lambda, \sqrt{\gamma^2 - \beta^2}}}{Z_{\Lambda,\gamma}} \approx \left(1 - \beta^2/\gamma^2\right)^{\frac{(m+n+1)d+2}{2}}
$$

For the condition $\mathsf{HighBits}(\mathbf{w}, \gamma_2) \neq \mathsf{HighBits}(\mathbf{w} + (-1)^b\mathbf{j}_n c, \gamma_2)$, the term $(-1)^b\mathbf{j}_n c$ affects only one bit, so the rejection rate for a single coefficient is $1/\gamma_2$. Since c has τ non-zero terms and the degree of $\mathbf{j}_n$ is n, we have

$$
P_2 = (1 - 1/\gamma_2)^{n\tau}
$$

The acceptance rate of the signature algorithm is

$$P_1 \cdot P_2 = \left(1 - \beta^2/\gamma^2\right)^{\frac{(m+n+1)d+2}{2}} \cdot (1 - 1/\gamma_2)^{n\tau}$$

□

3.6 Pseudocode Description

The pseudocode of the BiT signature scheme is shown in Figure 7, with the adopted functions summarized in Table 4. We add a norm check on $\mathbf{h}$ at Step 19 of the signing procedure to reject a small fraction of oversized hints for storage space. The chosen bound B_h ensures that the rejection rate in this step is less than 1%.

H	Instantiated as SM3 or FIPS 202
SampleInBall	Maps a seed $\tilde{c}$ to a polynomial $c \in \mathcal{R}_q$
ExpandA	Maps a seed ρ to a matrix $\mathbf{A} \in \mathcal{R}_q^{m \times n}$ in NTT domain representation
ExpandS	Maps a seed $\rho' \in \{0,1\}^\kappa$ to a pair $(\mathbf{s}_0, \mathbf{e}) \in S_\eta^m \times S_\eta^n$
ExpandMask	Maps a seed $seed_y$ and a counter value counter to $\mathbf{y} \in S_{\gamma_1}^m$

Table 4: Function Declaration

3.7 Security for 512-bit

For the 512-bit parameterization, the original scheme binds the message to the challenge through a two-stage pre-hash, i.e. $\mu = H(tr \parallel M)$ and $\widetilde{c} = H(\mathbf{w}_1||\mu)$. This construction limits the achievable security to the collision resistance of the underlying hash: the standard SM3 is only a 256-bit hash, far below the intended 512-bit target. Under a fixed public key, an adversary can therefore find two distinct messages M_0, M_1 with $H(tr \parallel M_0) = H(tr \parallel M_1)$, request a signature on M_0 and transfer it unchanged to M_1, since the reconstructed commitment and the message representative μ coincide and hence so does the challenge.

To remove this weakness we modify only the 512-bit case so that the challenge directly includes the raw message, i.e. $\widetilde{c} = H(tr \parallel \text{LE64}(|M|) \parallel M \parallel \mathbf{w}_1$. Now even if two messages share the same μ, their raw messages differ and the full challenge inputs no longer coincide: given a signature σ with its fixed commitment encoding $\mathbf{w}_\sigma$, transferring it unchanged requires an $M_1 \neq M_0$ satisfying $H(tr \parallel \text{LE64}(|M_1|) \parallel M_1 \parallel \mathbf{w}_\sigma) = H(tr \parallel \text{LE64}(|M_0|) \parallel M_0 \parallel \mathbf{w}_\sigma)$. Thus the transfer attack is converted from finding a message-pre-hash collision before signing into finding a second preimage of the fixed complete challenge after a signature is obtained, and a collision precomputed against an attacker-chosen commitment cannot be transferred directly because the signer's commitment is not guaranteed to equal it. As a result, SHA3-512 offers 256-bit collision resistance and 512-bit second-preimage resistance, so this fix blocks the old collision-based transfer. The implementation changes correspond to `bit_challenge_init`/`bit_challenge_hash` in `symmetric.inc` and the 512-bit Sign/Verify calls in `sign.inc`.

KGen(1^κ)

1 : $\zeta \leftarrow \{0,1\}^\kappa$
2 : $(\rho, \rho', K) \in \{0,1\}^\kappa \times \{0,1\}^{2\kappa} \times \{0,1\}^\kappa := \mathsf{H}(\zeta)$
3 : $\mathbf{A}_0 \in \mathcal{R}_q^{n \times m} := \mathsf{ExpandA}(\rho)$
4 : $(\mathbf{s}_0, \mathbf{e}) \in S_\eta^m \times S_\eta^n := \mathsf{ExpandS}(\rho')$
5 : if $\mathcal{N}(\mathbf{s}) > d\beta^2/\tau$, go into step 1
6 : $\mathbf{b} := \mathbf{A}_0\mathbf{s}_0 + \mathbf{e}$
7 : $tr := \mathsf{H}(\rho, \mathbf{b})$
8 : return $(\mathsf{pk}, \mathsf{sk}) := ((\rho, \mathbf{b}), (K, tr, \mathbf{s}_0, \mathbf{e}, \mathbf{b})$

Sign(sk, M)

1 : $\mathbf{A}_0 \in \mathcal{R}_q^{n \times m} := \mathsf{ExpandA}(\rho)$
2 : $\mathbf{A} := (\hat{q}\mathbf{j}_n - \mathbf{b}, \mathbf{A}_0, \mathbf{I}_n)$ $\hat{q} = 2^{-1} \bmod q$
3 : $\mathbf{s} := (1, \mathbf{s}_0, \mathbf{e})$
4 : $\mu \in \{0,1\}^\kappa := \mathsf{H}(tr||M)$
5 : $\mathsf{counter} := 0$
6 : $\rho'' \in \{0,1\}^\kappa := \mathsf{H}(K||\mu)$
7 : $\mathbf{y} \in P_{\gamma_1} := \mathsf{ExpandMask}(\rho'', \mathsf{counter})$
8 : $\mathbf{w} := \mathbf{A}\mathbf{y}$; $\mathsf{counter} := \mathsf{counter} + m$
9 : $\mathbf{w}_1 := \mathsf{HighBits}(\mathbf{w}, \gamma_2)$
10 : $\tilde{c} \in \{0,1\}^\kappa := \mathsf{H}(\mathbf{w}_1||\mu)$
11 : $c \in B_\tau := \mathsf{SampleInBall}(\tilde{c})$
12 : $b \in \{0,1\} := \mathsf{Sample}_b(\rho'')$
13 : $\mathbf{z} := \mathbf{y} + (-1)^b\mathbf{s}c$
14 : $\mathbf{z} := \mathsf{RejectSample}(\mathbf{z}, \gamma_1, \mathbf{s}c, \beta)$
15 : $(\mathbf{z}_{1,s_0}, \mathbf{z}_e) := \mathsf{Split}(\mathbf{z}) \in \mathcal{R}_q^{m+1} \times \mathcal{R}_q^n$
16 : if $\mathsf{HighBits}(\mathbf{w}, \gamma_2) \neq \mathsf{HighBits}(\mathbf{w} + (-1)^b\mathbf{j}_n c, \gamma_2)$, go into step 7
17 : $\hat{\mathbf{w}} = \mathbf{w} - \mathbf{z}_e + (1-b)\mathbf{j}_n c$
18 : $\mathbf{h} = \mathsf{HighBits}(\mathbf{w}, \gamma_2) - \mathsf{HighBits}(\hat{\mathbf{w}}, \gamma_2)$
19 : if $||\mathbf{h}||_\infty \geq B_{h_\infty}$ or $||\mathbf{h}||_2 \geq B_{h_2}$, go into step 7
20 : if $||(\mathbf{z}_{1,s_0}, \gamma_2\mathbf{h})||_2 > B_2$ or $||(\mathbf{z}_{1,s_0}, \gamma_2\mathbf{h})||_\infty > B_\infty$, go into step 7
21 : **return** $\sigma := (\mathbf{z}_{1,s_0}, \mathbf{h}, \tilde{c})$

Verify$(\mathsf{pk}, M, \sigma = (\mathbf{z}_{1,s_0}, \mathbf{h}, \tilde{c}))$

1 : $\mathbf{A}_0 \in \mathcal{R}_q^{n \times m} := \mathsf{ExpandA}(\rho)$
2 : $\mathbf{A}_{1,s_0} := (\hat{q}\mathbf{j}_n - \mathbf{b}, \mathbf{A}_0)$
3 : $tr := \mathsf{H}(\rho, \mathbf{b})$
4 : $\mu \in \{0,1\}^\kappa := \mathsf{H}(tr||M)$
5 : $c := \mathsf{SampleInBall}(\tilde{c})$
6 : $\hat{\mathbf{w}}' = \mathbf{A}_{1,s_0}\mathbf{z}_{1,s_0} + \hat{q}\mathbf{j}_n c$
7 : $\mathbf{w}' = \mathsf{HighBits}(\hat{\mathbf{w}}', \gamma_2) + \mathbf{h}$
8 : return $[\![||(\mathbf{z}_1, \gamma_2\mathbf{h})||_2 \leq B_2]\!]$ and $[\![||(\mathbf{z}_1, \gamma_2\mathbf{h})||_\infty \leq B_\infty]\!]$ and $[\![\tilde{c} = \mathsf{H}(\mathbf{w}', \mu)]\!]$

Figure 7: Pseudocode of BiT

3.8 NTRU Version

The description of the BiT signature scheme (NTRU Version) is shown in Figure 8.

KGen(1^κ)

1 : $f, g \xleftarrow{\$} S_\eta \times S_\eta \subset \mathcal{R}_q \times \mathcal{R}_q$
2 : if f is not invertible in $\mathcal{R}_q$, go into step 1
3 : $s = (f, -g)$
4 : $\hat{a} = (g + \hat{q})/f \in \mathcal{R}_q$
5 : $a := (\hat{a}, 1) \in \mathcal{R}_q^2$
6 : $(\mathsf{pk}, \mathsf{sk}) := (a, s)$

Verify$(\mathsf{pk}, \sigma, \mu)$

1 : if $||(z_1, \gamma_2 h)||_\infty > B_\infty$, return reject
2 : $\hat{w}' = \hat{a}z_1 + \hat{q}c \mod q$
3 : $w' = \mathsf{HighBits}(\hat{w}', \gamma_2) + h$
4 : Accept if $\mathbf{c} = \mathsf{H}(w', \mu)$

Sign$(\mathsf{pk}, \mathsf{sk}, \mu)$

1 : $y = (y_1, y_2) \xleftarrow{\$} P_{\gamma_1}$
2 : $w := ay \mod q$
3 : $c = \mathsf{H}(\mathsf{HighBits}(w, \gamma_2), \mu)$
4 : $b \xleftarrow{\$} \{0, 1\}$
5 : $z = (z_1, z_2) := y + (-1)^b sc$
6 : $z := \mathsf{RejectSample}(z, \gamma_1, sc, \beta)$
7 : if $z = \perp$, go into step 1
8 : if $\mathsf{HighBits}(w, \gamma_2) \neq \mathsf{HighBits}(w + (-1)^b \mathbf{c}, \gamma_2)$, go into step 1
9 : $\hat{w} = w - z_2 + (1 - b)c$
10 : $h = \mathsf{HighBits}(w, \gamma_2) - \mathsf{HighBits}(\hat{w}, \gamma_2)$
11 : if $||(z_1, \gamma_2 h)||_2 > B_2$ or $||(z_1, \gamma_2 h)||_\infty > B_\infty$, then restart
12 : **return** $\sigma := (z_1, h, c)$

Figure 8: BiT (NTRU Version)

4 Design Rationale

This section first introduces the design principles of BiT and the underlying hard mathematical problems. It then describes the parameter design criteria that balance security and efficiency, and finally presents the rules for selecting the challenge entropy.

4.1 Design Principles and Hard Problems

4.1.1 Design Principles

This scheme follows the Fiat-Shamir with Abort paradigm. Under the random oracle (RO) model, it transforms an identification (ID) protocol into a signature scheme. A standard ID protocol consists of three rounds of interaction between the prover and the verifier. First, the prover sends a commitment w to the verifier. The verifier then returns a random challenge c, and the prover responds with z, forming the transcript (w, c, z). The verifier checks the validity of the transcript and outputs 1 if it is valid, and 0 otherwise.

This transformation is naturally suited to number-theoretic signature schemes. In these schemes, the signer can completely hide the secret term using a masking vector, so the signature distribution does not leak information about the secret key. In contrast, the security of lattice-based schemes relies on the hardness of finding short lattice vectors. As a result, the masking vector must also be short, making it impossible to completely hide the secret key.

To prevent statistical attacks on the signature distribution, a rejection sampling mechanism is introduced to ensure that the final signature distribution is independent of the secret key. Compared with the standard ID protocol, the transformed scheme adds a response algorithm, Rep_{Abort}, in the third step. This algorithm selectively outputs the response z or aborts, thereby

ID Scheme	Signature Scheme	Signature with Abort
Interactive	**Signing**	**Signing**
1: Prover $\xrightarrow{(w,\mathsf{st})}$ Verifier	1: $w \leftarrow \mathsf{Com}(\mathsf{sk})$	1: $w \leftarrow \mathsf{Com}(\mathsf{sk})$
2: Prover $\xleftarrow{c}$ Verifier	2: $c \leftarrow \mathsf{RO}(w, \mu)$	2: $c \leftarrow \mathsf{RO}(w, \mu)$
3: Prover $\xrightarrow{z}$ Verifier	3: $z \leftarrow \mathsf{Rep}(w, c)$	3: $z \leftarrow \mathsf{Rep}_{Abort}(w, c)$
4: Output (w, c, z)	4: Output (w, c, z)	4: If $z = \perp$, restart; otherwise, output (w, c, z)
Verification	**Verification**	**Verification**
1: Accept if $V(w, c, z) = 1$	1: Accept if $V(w, c, z) = 1$	1: Accept if $V(w, c, z) = 1$

Figure 9: Fiat-Shamir with Abort

removing the dependence of the signature distribution on the secret key. Here, $\perp$ denotes an abort. The components that differ between schemes are highlighted in the boxes. Figure 9 illustrates the complete Fiat-Shamir with Aborts transformation process.

Rejection Sampling: We have introduced rejection sampling and its basic principle in the Introduction. Here, we further explain its advantages.

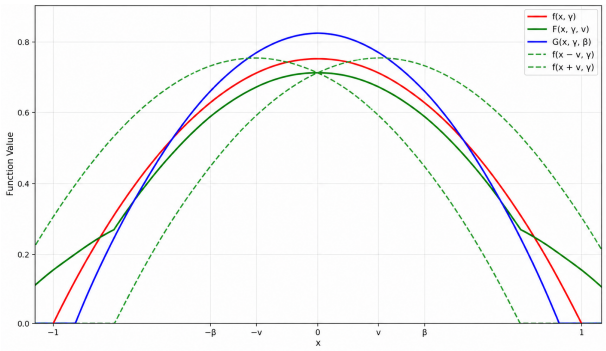


Figure 10: Rejection Sampling in BiT

As discussed in Section 3.5, letting γ and β denote the sizes of the masking term and the secret term, respectively, our acceptance rate is $(1 - \beta^2/\gamma^2)^{\frac{md+2}{2}}$. In comparison, ML-DSA, standardized by NIST, achieves an acceptance rate of $(1 - \beta^2/\gamma^2)^{md}$. Our rejection sampling achieves a quadratic improvement.

Signature Compression: The signing key can be decomposed into $\mathbf{s} = (\mathbf{s}_1, \mathbf{s}_2)$, and the signature is similarly decomposed into $(\mathbf{z}_1, \mathbf{z}_2)$. During verification, $\mathbf{z}_2$ appears only as an additive term and is never involved in multiplication. Moreover, $\mathbf{z}_2$ has a small norm and therefore has only a limited effect on the verification result. This property allows its influence to be further reduced by truncating the high-order bits of the computation results.

To eliminate the residual influence caused by high-order deviations, two mainstream approaches are commonly used. The first introduces an additional check during signing: if the high-order bits computed by the signer do not match those computed by the verifier, the current signature is rejected, and $\mathbf{z}_2$ is not output. However, this approach significantly increases the

rejection rate, reduces computational efficiency, and limits parameter selection and signature size. The second approach computes the difference between the high-order bits obtained by the signer and the verifier, and encodes it as auxiliary hint information $\mathbf{h}$. In essence, $\mathbf{h}$ compresses the original component $\mathbf{z}_2$, thereby significantly reducing the signature size. Since $\mathbf{h}$ is included in the signature, rejection sampling on $\mathbf{z}_2$ is still required to ensure security.

Since the optimized rejection sampling mechanism proposed in this work, the overhead introduced by the first approach is much higher than the cost of rejection sampling on $\mathbf{z}_2$. Therefore, this scheme adopts the second, hint-based compression approach to reduce the signature size efficiently.

4.1.2 Mathematical Hard Problems

The existential unforgeability of lattice-based signature schemes is typically reduced to the hardness of standard lattice problems, including the Shortest Vector Problem (SVP) and the Closest Vector Problem (CVP). In general, the security of a signature scheme relies on two components:

1. **Key generation security.** The public key must be computationally indistinguishable from a uniformly random element over the corresponding space. For MLWE-based schemes, this property relies on the hardness of the MLWE problem.

2. **Signature generation security.** The scheme must prevent an adversary from forging valid short signatures. This property is ultimately reduced to the hardness of the Module Short Integer Solution (MSIS) problem.

4.2 Criterion for Parameter Selection

Parameter selection is guided by two criteria: security and computational efficiency.

Specifically, the target security level determines the maximum entropy of the challenge polynomial $\mathbf{c}$, thereby limiting the value of parameter τ. The security of key generation and the unforgeability of signatures are respectively reduced to the hardness of the MLWE and MSIS problems. Core parameters, including the public key dimension and polynomial degree, are then chosen according to the complexity of these underlying problems, ensuring that all hardness assumptions meet or exceed the target security level. The detailed security reductions are presented in Section 5.

In addition, the overhead of rejection sampling is taken into account during parameter selection. In this work, the expected number of signing attempts required to generate a valid signature is approximately 1.5, providing a good balance between computational efficiency and security.

The parameters are respectively provided in Tables 5.

4.3 The post-quantum security for the entropy of the challenge

In this section, we present the criteria for setting the challenge entropy (i.e., the Hamming weight τ of $\boldsymbol{c}$) to ensure quantum security of the hash function under Grovers search with bounded circuit depth.

Table 5: Parameter Settings

Security Level	80	128	192	256	384	512
η [the density of 1/-1s in the key]	3/32	1/8	1/4	1/16	1/4	1/16
q [modulus]	3329	7681	7681	7681	25601	25601
d [order of the polynomial]	512	256	1024	512	2048	1024
(n, m) [dimension of pk]	(1, 1)	(3, 3)	(1,1)	(3, 3)	(1,1)	(3,3)
τ [hamming weight of c]	13	30	31	58	60	115
β [ℓ_2-norm of $\mathbf{s}c$ $(\eta\tau)$]	82	156	319	225	664	465
γ_1 [size of $\mathbf{y}$]	2875	8100	9000	15400	41600	66000
γ_2 [parameters for $\mathsf{HighBits}$]	256	375	320	470	1280	1900
B_2[the upper bound of $\|\|(\mathbf{z}, \gamma_2\mathbf{h})\|\|_2$]	3799	9469	10078	17735	48428	81158
B_{h_∞} [the upper bound of $\|\|\mathbf{h}\|\|_\infty$]	3	3	3	3	3	3
B_{h_2} [the upper bound of $\|\|\mathbf{h}\|\|_{\ell_2}$]	13	20	22	30	28	34
δ_κ [the security factor in Theorem 2]	1.3	1.3	1.3	1.3	1.3	1.3

4.3.1 Grover Search Algorithm

The Grover search algorithm (GSA) is a quantum algorithm for unstructured search. Given a search space of size $N = 2^n$, it finds a marked element in $O(\sqrt{N})$ iterations, providing a quadratic speedup over classical brute-force search.

Each Grover iteration consists of an oracle that marks the target element and a diffusion operator. Starting from a uniform superposition over the search space, if there are M marked elements, the probability of measuring a marked element is maximized after $\left\lfloor \frac{\pi}{4}\sqrt{\frac{N}{M}} \right\rfloor$ iterations. When $M \ll N$, this can be approximated by $\left\lfloor \frac{\pi}{4}\sqrt{N} \right\rfloor$.

4.3.2 Quantum Circuit Depth

The depth of a quantum circuit is the minimum number of sequential layers obtained by grouping all quantum gates that can be executed in parallel into the same layer. It determines the execution time of the circuit and is ultimately limited by the coherence time of the qubits.

To evaluate post-quantum security under realistic quantum computing assumptions, NIST defines three maximum quantum circuit depth thresholds, denoted by $\mathsf{MAXDEPTH}$. These

thresholds are listed in Table 6.

Table 6: MAXDEPTH

MAXDEPTH	**Description**
2^{40}	The approximate number of gates that presently envisioned quantum computing architectures are expected to serially perform in a year
2^{64}	The approximate number of gates that current classical computing architectures can perform serially in a decade
2^{96}	The approximate number of gates that atomic scale qubits with speed of light propagation times could perform in a millennium

4.3.3 Quantum Security Corresponding to Different Challenge Entropies

Let g and d denote the total number of quantum gates and the circuit depth of a single Grover iteration for a Grover search attack on SHA-3, respectively. Under the maximum quantum circuit depth constraint $\mathsf{MAXDEPTH}$, the number of Grover iterations is bounded by $\left\lfloor \frac{\mathsf{MAXDEPTH}}{d} \right\rfloor$. Accordingly, the searchable space satisfies $N \leq \left(\frac{4}{\pi}\frac{\mathsf{MAXDEPTH}}{d}\right)^2$. When the challenge entropy satisfies $2^k > \left(\frac{4}{\pi}\frac{\mathsf{MAXDEPTH}}{d}\right)^2$, the total number of quantum gates required for a Grover attack is

$$G = \frac{2^k}{N} \times g\frac{\pi}{4}\sqrt{N} > 2^k \left(\frac{\pi}{4}\right)^2 \frac{gd}{\mathsf{MAXDEPTH}}.$$

In each Grover iteration, the diffusion operator requires far fewer quantum gates and much smaller circuit depth than the SHA-3 oracle, so its cost can be neglected. According to the latest quantum resource estimation for SHA-3, $g = 2605074$ and $d = 1156$. Taking $\mathsf{MAXDEPTH} = 2^{64}$ as an example, the quantum security for different challenge entropies is shown in Table 7.

Table 7: Quantum Security for Different Challenge Entropies

Security Level	80	128	192	256	384	512
d [order of the polynomial]	512	256	1024	512	2048	1024
τ [hamming weight of c]	13	30	31	58	60	115
entropy of the challenge	84.24	129.74	196.68	256.81	386.61	514.33
quantum security	$2^{51.03}$	$2^{96.53}$	$2^{163.46}$	$2^{223.60}$	$2^{353.40}$	$2^{481.12}$

5 Security Statements and Analysis

5.1 Theoretical Security Analysis

The security analysis follows the standard Fiat–Shamir proof framework. We first establish standard $\mathsf{UF\text{-}CMA}$ security in two steps. In the first step, we reduce $\mathsf{UF\text{-}CMA}$ security to $\mathsf{UF\text{-}NMA}$

security, where the adversary is not allowed to make signing queries. In the second step, we reduce UF-NMA security to the hardness of the underlying lattice problems. We then address the additional case arising in the proof of SUF-CMA security, namely, when the adversary produces a valid forgery on a message that has already been queried to the signing oracle.

Theorem 2. *Let the* $\mathsf{MLWE}_{q,n,m,S_\eta,S_\eta}$ *problem be* $(t_{\mathsf{MLWE}}, \epsilon_{\mathsf{MLWE}})$*-hard, the search* $\mathsf{MSIS}_{q,n,(m+1),\gamma_{\mathsf{MSIS}}}$ *problem be* $(t_{\mathsf{MSIS}}, \epsilon_{\mathsf{MSIS}})$*-hard for* $\gamma_{\mathsf{MSIS}} = \delta_\kappa\sqrt{2B_2^2 + nd\gamma_2^2/12}$*, the* $\mathsf{BiomodalSelfTargetMSIS}_{q,n,m,\gamma_{\mathsf{Self}}}$ *problem be* $(t_{\mathsf{Self}}, \epsilon_{\mathsf{Self}})$*-hard for* $\gamma_{\mathsf{Self}} = \delta_\kappa\sqrt{B_2^2 + nd\gamma_2^2/12}$*, where* $\delta_\kappa > 1$ *is chosen such that the norm of the resulting solution exceeds* $\gamma_{\mathsf{Self}}/\gamma_{\mathsf{MSIS}}$ *only with negligible probability. Then BiT signature scheme in Figure 6 is* (t, ϵ, q_S, q_H)-SUF-CMA *secure in programmable random oracle model, where* $t \approx t_{\mathsf{MLWE}} + t_{\mathsf{MSIS}} + t_{\mathsf{Self}}$ *and*

$$\epsilon \leq \epsilon_{\mathsf{MLWE}} + \epsilon_{\mathsf{Self}} + \epsilon_{\mathsf{MSIS}} + negl$$

where $negl = \left[\kappa\rho^{-1}q_S(\kappa\rho^{-1}q_S + q_H) + q_S(q_S + q_H)\right]\left(\frac{2\gamma_2+1}{q}\right)^{nd} + q_S 2^{-\kappa}$.

Proof. The proof consists of three parts. First, in Lemma 2, the schemes CMA security is reduced to its NMA security. Next, in Lemma 3, we reduce the NMA security to the BiomodalSelfTargetMSIS problem. Finally, we establish the reduction from SUF-CMA to UF-CMA security under the MSIS assumption in Lemma 4. □

Lemma 2. *Let there be an algorithm* $\mathcal{A}$ *that* (t, ϵ, q_S, q_H)*- breaks* UF-CMA *security of the BiT signature scheme in Figure 6, the decision* $\mathsf{MLWE}_{q,n,m,S_\eta,S_\eta}$ *problem is* $(t_{\mathsf{MLWE}}, \epsilon_{\mathsf{MLWE}})$*-hard. Then, there also an algorithm that* $(t_{\mathsf{Game}}, \epsilon_{\mathsf{Game}}, q_S, q_H)$*-forges a signature for* Game *in Figure 11, where* $t_{\mathsf{Game}} \approx t + t_{\mathsf{MLWE}}$ *and*

$$\epsilon_{\mathsf{Game}} \geq \epsilon - negl - \epsilon_{\mathsf{MLWE}}$$

where $negl = \left[\kappa\rho^{-1}q_S(\kappa\rho^{-1}q_S + q_H) + q_S(q_S + q_H)\right]\left(\frac{2\gamma_2+1}{q}\right)^{nd} + q_S 2^{-\kappa}$.

Proof. To prove the lemma, we introduce intermediate games Game 1, Game 2, Game 3, Game 4 and Game as shown in Figure 11. Let ϵ_1 ($\epsilon_2, \epsilon_3, \epsilon_4, \epsilon_f$, respectively) denote the distinguishing advantage between Real and Game 1 (Game 1 and Game 2, Game 2 and Game 3, Game 3 and Game 4, Game 4 and Game, respectively). The boxed parts highlight the modifications from the previous game.

Real denotes the real signing algorithm. For simplicity, the rejection sampling procedures for $\mathbf{z}_{1,0}$, $\mathbf{z}_{1,1}$, and $\mathbf{z}_2$ are uniformly referred to as the rejection sampling for $\mathbf{z}$.

Real $\rightsquigarrow$ Game 1. In Game 1, when answering signing queries, the challenger no longer computes $c := \mathsf{H}(\mathsf{HighBits}(\mathbf{w}, \gamma_2), \mu)$. Instead, it samples $c \xleftarrow{\$} \mathcal{C}$ uniformly at random and later programs $\mathsf{H}(\mathsf{HighBits}(\mathbf{w}, \gamma_2), \mu) = c$ for each iteration.

Game 1 simulates RO in the real algorithm Real. The only difference is that, in Game 1, outputs are directly assigned to random oracle inputs, whereas in Real, the random oracle first checks whether the input has appeared before and returns the previously recorded output if so.

Clearly, Game 1 perfectly simulates Real as long as all random oracle inputs are distinct. Therefore, the distinguishing advantage introduced by this modification is bounded by the probability of random oracle input collisions. Since the adversary may query the same message μ

KGen : (1^κ) Real; Game 1; Game 2; Game 3; Game 4

$\mathbf{s}_0, \mathbf{e} \xleftarrow{\$} S_\eta^m \times S_\eta^n \subset \mathcal{R}_q^m \times \mathcal{R}_q^n$
$\mathbf{A}_0 \xleftarrow{\$} \mathcal{R}_q^{n \times m}$
$\mathbf{b} = \mathbf{A}_0\mathbf{s}_0 + \mathbf{e}$
$\mathbf{A} = [\hat{q}\mathbf{j}_n - \mathbf{b}, \mathbf{A}_0, \mathbf{I}_n] = [\mathbf{A}_{1,s_0}, \mathbf{I}_n]$
$\mathbf{s} := (1, \mathbf{s}_0^T, \mathbf{e}^T)^T$
if $\mathcal{N}(\mathbf{s}) > d\beta^2/\tau$, go into step 1
$\mathsf{pk} := \mathbf{A}; \mathsf{sk} := \mathbf{s}$

KGen : (1^κ) Game

$\mathbf{A}_0 \xleftarrow{\$} \mathcal{R}_q^{n \times m}$
$\mathbf{b} \xleftarrow{\$} \mathcal{R}_q^n$
$\mathbf{A} = [\hat{q}\mathbf{j}_n - \mathbf{b}, \mathbf{A}_0, \mathbf{I}_n] = [\mathbf{A}_{1,s_0}, \mathbf{I}_n]$
$\mathsf{pk} := \mathbf{A}$

Sign$(\mathsf{sk}, \mathsf{pk}, \mu)$ Real

$\mathbf{y} \xleftarrow{\$} P_{\gamma_1}$
$\mathbf{w} := \mathbf{A}\mathbf{y}$
$c \in \mathcal{R}_q := \mathsf{H}(\mathsf{HighBits}(\mathbf{w}, \gamma_2), \mu)$
$b \xleftarrow{\$} \{0,1\}$
$\mathbf{z} := \mathbf{y} + (-1)^b\mathbf{s}c$
$(\mathbf{z}_{1,s_0}, \mathbf{z}_e) := \mathsf{Split}(\mathbf{z}) \in \mathcal{R}_q^{m+1} \times \mathcal{R}_q^n$
$\boxed{\mathbf{z} := \mathsf{RejectSample}(\mathbf{z})}$
if $\mathsf{HighBits}(\mathbf{w}, \gamma_2) \neq \mathsf{HighBits}(\mathbf{w} + (-1)^b\mathbf{j}_n c, \gamma_2)$, go into step 1
$\hat{\mathbf{w}} = \mathbf{w} - \mathbf{z}_e + (1-b)\mathbf{j}_n c$
$\mathbf{h} = \mathsf{HighBits}(\mathbf{w}, \gamma_2) - \mathsf{HighBits}(\hat{\mathbf{w}}, \gamma_2)$
if $||(\mathbf{z}_{1,s_0}, \gamma_2\mathbf{h})||_2 > B_2$ or $||(\mathbf{z}_{1,s_0}, \gamma_2\mathbf{h})||_\infty > B_\infty$, then restart
return $\sigma := (\mathbf{z}_{1,s_0}, \mathbf{h}, c)$

Sign$(\mathsf{sk}, \mathsf{pk}, \mu)$ Game 1

$\mathbf{y} \xleftarrow{\$} P_{\gamma_1}$
$\boxed{c \xleftarrow{\$} \mathcal{C}}$
$b \xleftarrow{\$} \{0,1\}$
$\mathbf{z} := \mathbf{y} + (-1)^b\mathbf{s}c$
$(\mathbf{z}_{1,s_0}, \mathbf{z}_e) := \mathsf{Split}(\mathbf{z}) \in \mathcal{R}_q^{m+1} \times \mathcal{R}_q^n$
$\boxed{\mathbf{w} := \mathbf{A}\mathbf{y}}$
$\boxed{\text{program } c := \mathsf{H}(\mathsf{HighBits}(\mathbf{w}, \gamma_2), \mu)}$
$\mathbf{z} := \mathsf{RejectSample}(\mathbf{z})$
if $\mathsf{HighBits}(\mathbf{w}, \gamma_2) \neq \mathsf{HighBits}(\mathbf{w} + (-1)^b\mathbf{j}_n c, \gamma_2)$, go into step 1
$\hat{\mathbf{w}} = \mathbf{w} - \mathbf{z}_e + (1-b)\mathbf{j}_n c$
$\mathbf{h} = \mathsf{HighBits}(\mathbf{w}, \gamma_2) - \mathsf{HighBits}(\hat{\mathbf{w}}, \gamma_2)$
if $||(\mathbf{z}_{1,s_0}, \gamma_2\mathbf{h})||_2 > B_2$ or $||(\mathbf{z}_{1,s_0}, \gamma_2\mathbf{h})||_\infty > B_\infty$, then restart
return $\sigma := (\mathbf{z}_{1,s_0}, \mathbf{h}, c)$

Sign$(\mathsf{sk}, \mathsf{pk}, \mu)$ Game 2

$\mathbf{y} \xleftarrow{\$} P_{\gamma_1}$
$c \xleftarrow{\$} \mathcal{C}$
$b \xleftarrow{\$} \{0,1\}$
$\mathbf{z} := \mathbf{y} + (-1)^b\mathbf{s}c$
$(\mathbf{z}_{1,s_0}, \mathbf{z}_e) := \mathsf{Split}(\mathbf{z}) \in \mathcal{R}_q^{m+1} \times \mathcal{R}_q^n$
$\mathbf{z} := \mathsf{RejectSample}(\mathbf{z})$
$\boxed{\mathbf{w} := \mathbf{A}\mathbf{z} - (-1)^b\hat{q}\mathbf{j}_n c}$
if $\mathsf{HighBits}(\mathbf{w}, \gamma_2) \neq \mathsf{HighBits}(\mathbf{w} + (-1)^b\mathbf{j}_n c, \gamma_2)$, go into step 1
$\hat{\mathbf{w}} = \mathbf{w} - \mathbf{z}_e + (1-b)\mathbf{j}_n c$
$\mathbf{h} = \mathsf{HighBits}(\mathbf{w}, \gamma_2) - \mathsf{HighBits}(\hat{\mathbf{w}}, \gamma_2)$
if $||(\mathbf{z}_{1,s_0}, \gamma_2\mathbf{h})||_2 > B_2$ or $||(\mathbf{z}_{1,s_0}, \gamma_2\mathbf{h})||_\infty > B_\infty$, then restart
return $\sigma := (\mathbf{z}_{1,s_0}, \mathbf{h}, c)$
$\boxed{\text{program } c := \mathsf{H}(\mathsf{HighBits}(\mathbf{w}, \gamma_2), \mu)}$

Sign$(\mathsf{sk}, \mathsf{pk}, \mu)$ Game 3

$\boxed{(\mathbf{z}_{1,s_0}, \mathbf{z}_e) \xleftarrow{\$} P_{\sqrt{\gamma^1 - \beta^2}}}$
$c \xleftarrow{\$} \mathcal{C}$
$b \xleftarrow{\$} \{0,1\}$
$(\mathbf{z}_{1,s_0}, \mathbf{z}_e) := \mathsf{Split}(\mathbf{z}) \in \mathcal{R}_q^{m+1} \times \mathcal{R}_q^n$
$\mathbf{w} := \mathbf{A}\mathbf{z} - (-1)^b\hat{q}\mathbf{j}_n c$
if $\mathsf{HighBits}(\mathbf{w}, \gamma_2) \neq \mathsf{HighBits}(\mathbf{w} + (-1)^b\mathbf{j}_n c, \gamma_2)$, go into step 1
$\hat{\mathbf{w}} = \mathbf{w} - \mathbf{z}_e + (1-b)\mathbf{j}_n c$
$\mathbf{h} = \mathsf{HighBits}(\mathbf{w}, \gamma_2) - \mathsf{HighBits}(\hat{\mathbf{w}}, \gamma_2)$
if $||(\mathbf{z}_{1,s_0}, \gamma_2\mathbf{h})||_2 > B_2$ or $||(\mathbf{z}_{1,s_0}, \gamma_2\mathbf{h})||_\infty > B_\infty$, then restart
return $\sigma := (\mathbf{z}_{1,s_0}, \mathbf{h}, c)$
program $c := \mathsf{H}(\mathsf{HighBits}(\mathbf{w}, \gamma_2), \mu)$

Sign(pk, μ) Game 4; Game

$(\mathbf{z}_{1,s_0}, \mathbf{z}_e) \xleftarrow{\$} P_{\sqrt{\gamma_1^2 - \beta^2}}$
$c \xleftarrow{\$} \mathcal{C}$
$\boxed{\mathbf{w}_+ := \mathbf{A}\mathbf{z} + \hat{q}\mathbf{j}_n c}$
$\boxed{\mathbf{w}_- := \mathbf{A}\mathbf{z} - \hat{q}\mathbf{j}_n c}$
$\boxed{\text{if } \mathsf{HighBits}(\mathbf{w}_+, \gamma_2) \neq \mathsf{HighBits}(\mathbf{w}_-, \gamma_2), \text{ go into step 1}}$
$\boxed{\hat{\mathbf{w}} = \mathbf{w}_+ - \mathbf{z}_e}$
$\mathbf{h} = \mathsf{HighBits}(\mathbf{w}_+, \gamma_2) - \mathsf{HighBits}(\hat{\mathbf{w}}, \gamma_2)$
if $||(\mathbf{z}_{1,s_0}, \gamma_2\mathbf{h})||_2 > B_2$ or $||(\mathbf{z}_{1,s_0}, \gamma_2\mathbf{h})||_\infty > B_\infty$, then restart
return $\sigma := (\mathbf{z}_{1,s_0}, \mathbf{h}, c)$
$\boxed{\text{program } c := \mathsf{H}(\mathsf{HighBits}(\mathbf{w}_+, \gamma_2), \mu)}$

Figure 11: The Security Games

multiple times, we focus on the probability of $\mathsf{HighBits}(\mathbf{w}, \gamma_2) = \mathsf{HighBits}(\mathbf{w}', \gamma_2)$.

$$\begin{aligned}\Pr[\mathsf{HighBits}(\mathbf{w}, \gamma_2) = \mathsf{HighBits}(\mathbf{w}', \gamma_2)] &= \Pr[\mathsf{HighBits}(\mathbf{Ay}, \gamma_2) = \mathsf{HighBits}(\mathbf{Ay}', \gamma_2)] \\ &\leq \Pr[||\mathbf{Ay} - \mathbf{Ay}'||_\infty \leq \gamma_2] \\ &= \Pr[||\mathbf{u}||_\infty \leq \gamma_2 | \mathbf{u} \xleftarrow{\$} \mathcal{R}_q^n] = \left(\frac{2\gamma_2 + 1}{q}\right)^{nd}\end{aligned}$$

The inequality follows from the definition of $\mathsf{HighBits}$. Since $\mathsf{HighBits}$ extracts the high-order bits by performing modulo γ_2, two inputs whose difference exceeds γ_2 must produce different outputs under $\mathsf{HighBits}$. The second equation follows by heuristically modeling $\mathbf{A}(\mathbf{y} - \mathbf{y}')$ as a uniformly random element $\mathbf{u} \in \mathcal{R}_q^n$.

The probability of a single rewind in the scheme generating a valid signature is ρ. Excluding the probability of $q_S 2^{-\kappa}$, the maximum number of rejections in $\mathsf{Game}\ 1$ is κ/ρ. The number of defined points of the RO within $\mathsf{Game}\ 1$ is upper bounded by $\kappa\rho^{-1}q_S + q_H$. Additionally, there are at most $\kappa\rho^{-1}q_S$ reprogramming steps. Therefore, we have

$$\epsilon_1 \leq \kappa\rho^{-1}q_S(\kappa\rho^{-1}q_S + q_H)\left(\frac{2\gamma_2 + 1}{q}\right)^{nd} + q_S 2^{-\kappa} \tag{3}$$

Game 1 $\rightsquigarrow$ **Game 2**. In $\mathsf{Game}\ 2$, the generation of $\mathbf{w}$ is changed from $\mathbf{w} := \mathbf{Ay}$ to $\mathbf{w} := \mathbf{Az} - (-1)^b\hat{q}\mathbf{j}_n c$. It is straightforward to verify that $\mathbf{Az} - (-1)^b\hat{q}\mathbf{j}_n c = \mathbf{Ay}$, thus this modification introduces no distinguishing advantage.

Additionally, the programming position is modified. In $\mathsf{Game}\ 2$, the assignment $c := \mathsf{H}(\mathsf{HighBits}(\mathbf{w}, \gamma_2), \mu)$ is executed after the signature is output. Similar to the analysis in **Real** $\rightsquigarrow$ **Game 1**, $\mathsf{Game}\ 1$ and $\mathsf{Game}\ 2$ are identical as long as all random oracle inputs are distinct. Hence, the distinguishing advantage is bounded by the probability of random oracle input collisions, which is $\left(\frac{2\gamma_2+1}{q}\right)^{nd}$.

Note that in $\mathsf{Game}\ 2$, only output signatures are programmed into RO. Therefore, it suffices to consider collisions involving output signatures only. The adversary can issue at most q_S signing queries and q_H RO queries. Hence, the number of defined RO points in $\mathsf{Game}\ 2$ is bounded by $q_H + q_S$, and there are at most q_S output signatures. Therefore, we have $\epsilon_2 \leq q_S(q_S + q_H)\left(\frac{2\gamma_2+1}{q}\right)^{nd}$.

Game 2 $\rightsquigarrow$ **Game 3**. In $\mathsf{Game}\ 3$, the generation of the signature component $\mathbf{z}$ is modified. Instead of computing $\mathbf{z} = \mathbf{y} + (-1)^b\mathbf{s}c$, the challenger directly samples $\mathbf{z} \leftarrow P_{\sqrt{\gamma^2 - \beta^2}}$. The correctness of this replacement follows from the rejection sampling algorithm $\mathsf{RejectSample}$. We have $\epsilon_3 = 0$.

Game 3 $\rightsquigarrow$ **Game 4**. **Game 4** removes the auxiliary bimodal bit b. Define

$$\mathbf{w}_+ = \mathbf{Az} + \hat{q}\mathbf{j}_n c, \qquad \mathbf{w}_- = \mathbf{Az} - \hat{q}\mathbf{j}_n c.$$

Using $2\hat{q} = 1 \bmod q$, when $b = 0$ we have $(\mathbf{w}, \mathbf{w} + \mathbf{j}_n c) = (\mathbf{w}_-, \mathbf{w}_+)$, whereas when $b = 1$, $(\mathbf{w}, \mathbf{w} - \mathbf{j}_n c) = (\mathbf{w}_+, \mathbf{w}_-)$. Therefore, the rejection condition in Game 3 is equivalent, for either value of b, to $\mathsf{HighBits}(\mathbf{w}_+, \gamma_2) = \mathsf{HighBits}(\mathbf{w}_-, \gamma_2)$. Furthermore, $\hat{\mathbf{w}} = \mathbf{w} - \mathbf{z}_e + (1-b)\mathbf{j}_n c = \mathbf{w}_+ - \mathbf{z}_e$ for both values of b. Conditioned on passing the rejection check, $\mathsf{HighBits}(\mathbf{w}, \gamma_2) = \mathsf{HighBits}(\mathbf{w}_+, \gamma_2)$, and

hence $\mathbf{h} = \mathsf{HighBits}(\mathbf{w}_+, \gamma_2) - \mathsf{HighBits}(\mathbf{w}_+ - \mathbf{z}_e, \gamma_2)$. The random-oracle programming point is likewise $(\mathsf{HighBits}(\mathbf{w}_+, \gamma_2), \mu)$.

Thus, b has no influence on the restart probability, the hint, the norm checks, the programmed random-oracle point, or the output signature. Removing it is therefore a purely algebraic rewriting, and Game 3 and Game 4 are identically distributed. Hence, $\epsilon_4 = 0$.

Game 4 $\rightsquigarrow$ Game. Game modifies the generation of the verification key. In Game 4, $(\mathbf{A}_0, \mathbf{b})$ is sampled as an MLWE instance, and the verification key $\mathbf{A}$ is derived from it. In contrast, in Game, $(\mathbf{A}_0, \mathbf{b})$ is sampled uniformly from $\mathcal{R}_q^{n\times m} \times \mathcal{R}_q^n$. The difference between these two games is bounded by the MLWE assumption. Therefore, $\epsilon_f \leq \epsilon_{\mathsf{MLWE}}$.

According to the analysis above, we have $\epsilon_{\mathsf{Game}} \geq \epsilon - \epsilon_{\mathsf{MLWE}} - negl$, where $negl = [\kappa\rho^{-1}q_S(\kappa\rho^{-1}q_S + q_H) + q_S(q_S + q_H)]\left(\frac{2\gamma_2+1}{q}\right)^{nd} + q_S 2^{-\kappa}$ □

Lemma 3. *Let there be an adversary $\mathcal{F}$ that $(t_{\mathsf{Game}}, \epsilon_{\mathsf{Game}}, q_S, q_H)$-forges a signature in* Game *in Figure 11, then there is also an adversary $\mathcal{F}'$ with running time $t_{\mathsf{Self}} \approx t_{\mathsf{Game}}$ can solve the $\mathsf{BimodalSelfTargetMSIS}_{q,n,m,\gamma_{\mathsf{Self}}}$ problem for $\gamma_{\mathsf{Self}} = \delta_\kappa\sqrt{B_2^2 + nd\gamma_2^2/12}$, where $\delta_\kappa > 1$ is chosen such that the norm of the resulting solution exceeds γ_{Self} only with negligible probability. The probability ϵ_{Self} is lower bounded by: $\epsilon_{\mathsf{Self}} \geq \epsilon_{\mathsf{Game}}$.*

Proof. Let $\mathcal{F}$ successfully forges the signature $(\mathbf{z}_1, \mathbf{h}, c)$ on message μ. Then

$$\mathsf{H}(\mathsf{HighBits}(\mathbf{A}_{1s_0}\mathbf{z}_{1s_0} + \hat{q}\mathbf{j}_n c, \gamma_2) + \mathbf{h}, \mu) = c$$

Then, there is

$$\gamma_2 \cdot [\mathsf{HighBits}(\mathbf{A}_{1s_0}\mathbf{z}_{1s_0} + \hat{q}\mathbf{j}_n c, \gamma_2) + \mathbf{h}] = \mathbf{A}_{1s_0}\mathbf{z}_{1s_0} + \hat{q}\mathbf{j}_n c + \gamma_2\mathbf{h} + \mathbf{u}$$

where $||\mathbf{u}||_\infty \leq \gamma_2/2 + 1$ comes from removal operation for HighBits. Then

$$\mathsf{H}(\frac{1}{\gamma_2}(\mathbf{A}_{1s_0}\mathbf{z}_{1s_0}1 + \hat{q}\mathbf{j}_n c + \gamma_2\mathbf{h} + \mathbf{u}), \mu) = c$$

Express the equation as a multiplication involving matrices and vectors:

$$\mathsf{H}(\frac{1}{\gamma_2}([\mathbf{A}_{1s_0}|\mathbf{I}] \cdot \begin{bmatrix} \mathbf{z}_{1s_0} \\ \gamma_2\mathbf{h} + \mathbf{u} \end{bmatrix} + \hat{q}\mathbf{j}_n c), \mu) = c$$

Define $\mathsf{H}'(\gamma_2\mathbf{x}, \mu) = \mathsf{H}(\mathbf{x}, \mu)$, then

$$\mathsf{H}'([\mathbf{A}_{1s_0}|\mathbf{I}] \cdot \begin{bmatrix} \mathbf{z}_{1s_0} \\ \gamma_2\mathbf{h} + \mathbf{u} \end{bmatrix} + \hat{q}\mathbf{j}_n c, \mu) = c$$

Let $\mathbf{y} = \begin{bmatrix} \mathbf{z}_{1,s_0} \\ \gamma_2\mathbf{h} + \mathbf{u} \end{bmatrix}$. By the norm check, we have $\left\| \begin{bmatrix} \mathbf{z}_{1,s_0} \\ \gamma_2\mathbf{h} \end{bmatrix} \right\|_2 \leq B_2$. Moreover, we have $\mathbb{E}[\|\mathbf{u}\|_2^2] = nd\gamma_2^2/12$. Therefore, the natural scale of $\|\mathbf{y}\|_2$ is $\sqrt{B_2^2 + nd\gamma_2^2/12}$. Let δ_κ denote a security factor chosen according to the target security level such that the probability that $\|\mathbf{y}\|_2$ exceeds

$\delta_\kappa\sqrt{B_2^2 + nd\gamma_2^2/12}$ is negligible. Hence, except with negligible probability,

$$\|\mathbf{y}\|_2 \leq \delta_\kappa\sqrt{B_2^2 + \frac{nd\gamma_2^2}{12}}.$$

Thus, $\mathbf{y}$ satisfies the required norm bound and constitutes a valid solution to the BimodalSelfTargetMSIS problem. □

Lemma 4. *Let there be an adversary $\mathcal{F}$ that $(t_{\mathsf{SUF}}, \epsilon_{\mathsf{SUF}}, q_S, q_H)$-forges another signature for the message that already has an existing signature in the scheme in 6, then there is also an adversary $\mathcal{F}'$ with running time $t_{\mathsf{MSIS}} \approx t_{\mathsf{SUF}}$ can solve the $\mathsf{MSIS}_{q,n,(m+1),\gamma_{\mathsf{MSIS}}}$ problem for parameters $n, m, q \in \mathbb{N}$, $\gamma_{\mathsf{MSIS}} = \delta_\kappa\sqrt{2B_2^2 + nd\gamma_2^2/12}$ and the ring $\mathcal{R}_q$, where $\delta_\kappa > 1$ is chosen such that the norm of the resulting solution exceeds γ_{MSIS} only with negligible probability. The probability ϵ_{MSIS} is lower bounded by: $\epsilon_{\mathsf{MSIS}} \geq \epsilon_{\mathsf{SUF}}$.*

Proof. The SUF-CMA security is guaranteed by the hardness of the MSIS problem. To prove SUF-CMA security, the adversary $\mathcal{F}$ is allowed to make signing queries and must output a new signature $(\mathbf{z}'_{1,s_0}, \mathbf{h}', c)$ for a message μ with an existing signature $(\mathbf{z}_{1,s_0}, \mathbf{h}, c)$, where the challenge c remains unchanged by the forking lemma.

$$\mathsf{H}(\mathsf{HighBits}(\mathbf{A}_{1,s_0}\mathbf{z}'_{1,s_0} + \hat{q}\mathbf{j}_n c, \gamma_2) + \mathbf{h}' \bmod q, \mu) = c$$

$$\mathsf{H}(\mathsf{HighBits}(\mathbf{A}_{1,s_0}\mathbf{z}_{1,s_0} + \hat{q}\mathbf{j}_n c, \gamma_2) + \mathbf{h} \bmod q, \mu) = c$$

According to the collision resistance of H, there is

$$\begin{aligned}
\mathsf{HighBits}(\mathbf{A}_{1,s_0}\mathbf{z}'_{1,s_0} + \hat{q}\mathbf{j}_n c, \gamma_2) + \mathbf{h}' &= \mathsf{HighBits}(\mathbf{A}_{1,s_0}\mathbf{z}_{1,s_0} + \hat{q}\mathbf{j}_n c, \gamma_2) + \mathbf{h} \\
\mathbf{A}_{1,s_0}\mathbf{z}'_{1,s_0} + \hat{q}\mathbf{j}_n c + \gamma_2\mathbf{h}' &= \mathbf{A}_{1,s_0}\mathbf{z}_{1,s_0} + \hat{q}\mathbf{j}_n c + \gamma_2\mathbf{h} + \mathbf{u} \\
[\mathbf{A}_{1,s_0}|\mathbf{I}] \cdot \begin{bmatrix} \mathbf{z}_{1,s_0} - \mathbf{z}'_{1,s_0} \\ \gamma_2(\mathbf{h} - \mathbf{h}') + \mathbf{u} \end{bmatrix} &= 0 \\
[\mathbf{A}_{1,s_0}|\mathbf{I}] \cdot \mathbf{y} &= 0
\end{aligned}$$

where $\mathbf{u}$ arises from removing HighBits. Following an analysis similar to that in Lemma 3, we obtain

$$\|\mathbf{y}\|_2 \leq \delta_\kappa\sqrt{2B_2^2 + \frac{nd\gamma_2^2}{12}},$$

where $\delta_\kappa > 1$ is chosen such that the probability that the norm of the resulting solution exceeds γ_{MSIS} is negligible. Thus we obtain a solution of $\mathsf{MSIS}_{q,n,m,\gamma_{\mathsf{MSIS}}}$ problem. □

5.2 Practical Security

Following the methodology of the NIST PQC standardization documents for lattice-based signatures, our concrete security evaluation considers both key-recovery and signature-forgery attacks. Key recovery corresponds to the decisional MLWE problem induced by the public key, while forgery security is based on SIS-type problems appearing in the reductions of Section 3.2.

Specifically, UF-CMA security reduces to the hardness of MLWE and BimodalSelfTargetMSIS, whereas SUF-CMA security additionally relies on the hardness of the corresponding MSIS problem.

We estimate the hardness of structured module/ring problems by treating them as the corresponding unstructured lattice problems over $\mathbb{Z}_q$. This provides a common and conservative basis for comparison with other lattice-based submissions, avoiding reliance on unproven additional hardness from the module or ring structure. In particular, the MLWE and MSIS instances are evaluated as ordinary LWE and SIS instances of the corresponding dimensions.

We used the lattice-estimator [1, 2] to evaluate all proposed parameter sets. The estimator implements the standard LWE and SIS attack families, including primal, dual, and hybrid attacks. In our analysis, all relevant attacks are enabled and compared under the same lattice-instance parameters. BKZ costs were estimated using the core-SVP model (`ADPS16` in the estimator), which measures the cost of BKZ reduction by the cost of solving exact SVP subproblems in the BKZ blocks [3, 4]. Specifically, once the estimator determines the BKZ block size β for a successful attack, classical and quantum costs are derived from the best-known asymptotic costs of lattice sieving in dimension β. This simple model enables direct comparison with earlier estimates based on BKZ plus classical or quantum sieving.

Table 8: Practical Security (MLWE Version)

Security Level	80	128	192	256	384	512
MLWE Hardness (Core-SVP)						
BKZ block-size	283	439	681	907	1353	1816
Classical	82	128	199	264	395	531
Quantum	76	117	182	243	362	488
MSIS Hardness (Core-SVP)						
BKZ block-size (SUF)	384(274)	560(442)	811(658)	1183(880)	1587(1317)	2379(1802)
Classical (SUF)	112(80)	163(129)	236(192)	345(256)	463(384)	694(526)
Quantum (SUF)	101 (72)	148(117)	214(174)	313(233)	420(349)	630(477)

5.3 Security Against Physical Attacks

Compared with ML-DSA [11], BiT reduces the risk of several power-based side-channel attacks through its design.

- **NTT-domain secret multiplication.** BiT does not compute secret-challenge products such as $\mathbf{s}c$ and $\mathbf{e}c$ by point-wise multiplication in the NTT domain. This follows from the parameter selection and the structure of the challenge polynomial c, where sparse multiplication is more efficient than NTT-based multiplication. As a result, attacks targeting secret-dependent NTT operations in ML-DSA, such as $c\mathbf{s}_1$ or $c\mathbf{s}_2$, cannot be directly applied to BiT [6, 14, 9, 12].

- **Reduced leakage from intermediate values.** BiT mainly uses the high-order bits $\mathbf{w}_1 := \mathsf{HighBits}(\mathbf{w}, \gamma_2)$ during signing and reduces the use of intermediate values such as the low-order bits $\mathbf{w}_0$. Therefore, attacks that rely on leakage from these intermediate values are not directly applicable to BiT [5].

- **Additional protection from the bimodal structure.** In ML-DSA, side-channel attacks often exploit the relation

$$\mathbf{z} := \mathbf{y} + c\mathbf{s}_1,$$

where leakage from $\mathbf{y}$ (such as zero coefficients or partial coefficient bits) or from rejected signatures $(\mathbf{z}, c)$ may reveal information about $c\mathbf{s}_1$ [10, 13, 16].

In BiT, this computation is replaced by

$$\mathbf{z}_{1,0} := \mathbf{y}_{1,0} + (-1)^b c, \quad \mathbf{z}_{1,1} := \mathbf{y}_{1,1} + (-1)^b \mathbf{s}_0 c, \quad \mathbf{z}_2 := \mathbf{y}_2 + (-1)^b \mathbf{e} c.$$

Here, $\mathbf{z}_{1,0}$ is independent of the secret vectors $\mathbf{s}_0$ and $\mathbf{e}$, and $\mathbf{z}_2$ is not included in the final signature. The remaining component,

$$\mathbf{z}_{1,1} := \mathbf{y}_{1,1} + (-1)^b \mathbf{s}_0 c,$$

also depends on the secret bit b. Therefore, even if similar side-channel leakage is available, the key-recovery techniques developed for ML-DSA cannot be directly applied to recover $\mathbf{s}_0 c$ without also recovering b.

6 Performance Evaluation

6.1 Performance Analysis

In BiT, the signing process consists of the following main steps: sampling the masking vector $\mathbf{y}$, computing $\mathbf{Ay}$, generating the challenge $\mathbf{c}$, performing rejection sampling on $\mathbf{z}$, and producing the hint $\mathbf{h}$.

- **Sampling y:** We sample from a parabolic distribution and provide an efficient implementation whose performance is comparable to that of state-of-the-art Gaussian samplers.

- **Computing Ay:** All computations are performed in the NTT domain, and fully split polynomial rings are used. The efficiency is comparable to mainstream schemes.

- **Generating the challenge c:** The challenge is generated using two hash functions, following the same structure as mainstream lattice-based signature schemes.

- **Computing $c\mathbf{s}$:** The challenge c is sparse, with Hamming weight τ and coefficients in $\{0, 1\}$. The products $c\mathbf{s}$, $c\mathbf{e}$, and $c\mathbf{b}_0$ are therefore computed by sparse multiplication instead of the NTT, which is fast because τ is small and the nonzero entries are all 1, so only additions are required.

- **Rejection sampling on z:** We use a simple rejection sampling method. Most candidate signatures $\mathbf{z}$ can be verified by checking their norms only, while a small fraction require an additional coin-flipping step to decide acceptance.
- **Producing the hint h:** The generation of $\mathbf{h}$ has similar cost to mainstream schemes.
- **Optimization notes:** The improved rejection sampling allows smaller parameters. For example, BiT uses modulus 7681, while ML-DSA uses 8380417 at the 128-bit security level. As a result, polynomial matrix–vector multiplication in BiT is significantly faster, making the scheme more efficient than ML-DSA.

6.2 Performance Test

6.2.1 x86 Platforms

Reference Implementation: The performance of the reference implementation was evaluated using the C implementation of BiT. To assess the impact of different symmetric components on the overall running time, we report two sets of timing results: one instantiated with SM3 and the other instantiated with FIPS202. All benchmarks were executed on an Intel Core i7-8750H processor running at 2.2 GHz, with Turbo Boost and hyper-threading disabled to reduce measurement variance. The test platform was equipped with 16 GiB of memory and ran Ubuntu 24.04.2 LTS. The implementation was built using GNU Make 4.3 and compiled with GNU GCC 13.3.0 using the flags `-std=c99`, `-Wpedantic`, `-Wall`, `-Wextra`, and `-O2`. Each reported value represents the average number of CPU cycles over 100000 independent runs.

Optimized Implementation with AVX2: The performance of the optimized implementation was evaluated using the x86 AVX2 implementation of BiT. As before, we report two sets of timing results to assess the impact of the symmetric component. The first set corresponds to the SM3-based instantiation, where AVX2 optimizations are applied to all operations except the symmetric component, while SM3 itself remains implemented in standard C; since we do not provide a dedicated AVX2 implementation of SM3, these results include the cost of the scalar SM3 component and are therefore less competitive than a fully vectorized symmetric instantiation. The second set corresponds to the FIPS202-based instantiation, for which a four-way parallel AVX2 implementation of the symmetric component is provided.

6.2.2 Performance Test Results

- Table 9 presents the efficiency and size results for the SM3-based implementation. Table 10 supplements these results with efficiency results for the FIPS 202-based implementation.
- The results are reported for the 128-bit, 256-bit, and 512-bit security levels, including both AVX and REF implementations. The results for other security levels are given in Section 6.3.

Table 9: Efficiency and Size of SM3-Based Implementation

Security Level	128	256	512
Efficiency AVX (clcyes)			
KGen	606,380	1,433,713	4,898,615
Sign	4,050,600	8,932,129	25,491,662
Verify	356,020	847,968	3,418,866
Efficiency REF (clcyes)			
KGen	827,208	1,822,733	5,574,446
Sign	4,858,681	10,889,171	29,133,219
Verify	614,801	1,412,218	4,210,340
Size (bytes)			
pk	1264	2528	5824
sig	1440	2976	7474
pk+sig	2704	5504	13298

Table 10: Efficiency of FIPS 202-Based Implementation

Security Level	128	256	512
Efficiency AVX (clcyes)			
KGen	295,318	749,930	1,657,675
Sign	435,985	952,465	2,127,980
Verify	95,723	238,804	688,667
Efficiency REF (clcyes)			
KGen	576,296	1,424,450	2,751,671
Sign	1,883,694	4,354,218	7,769,872
Verify	389,547	887,285	1,736,667

- Transmission and Storage Cost: The transmission and storage cost is measured by the sizes of the public key, secret key, and signature. Since the public key and signature are usually transmitted together, the combined size $\mathsf{pk} + \mathsf{sig}$ is also reported.

6.3 Parameter, Security, and Size Summary

Table 11 summarizes the parameter settings, security levels, and size metrics of the BiT signature schemes.

7 Feature Statements

The proposed scheme is designed according to the following principles.

Hardness of underlying assumptions: The scheme is based on MLWE problem and MSIS problem. The hardness of these problems has been extensively studied and is widely accepted, providing a solid security foundation for the proposed scheme.

Conservative parameter selection: To ensure long-term security, the scheme adopts a conservative parameter selection strategy. The parameters are chosen so that the corresponding **core-SVP** problem remains sufficiently hard, providing resilience against potential future attacks.

Table 11: Parameter, Security, Size

Security Level	80	128	192	256	384	512
η	3/32	1/8	1/4	1/16	1/4	1/16
q	3329	7681	7681	7681	25601	25601
d	512	256	1024	512	2048	1024
(n, m)	(1, 1)	(3, 3)	(1,1)	(3, 3)	(1,1)	(3,3)
τ	13	30	31	58	60	115
β	82	156	319	225	664	465
γ_1	2875	8100	9000	15400	41600	66000
γ_2	256	375	320	470	1280	1900
B_2	3799	9469	10078	17735	48428	81158
B_{h_∞}	3	3	3	3	3	3
B_{h_2}	13	20	22	30	28	34
δ_κ	1.3	1.3	1.3	1.3	1.3	1.3
acceptance rate	50%	61%	13%	53%	43%	74%
MLWE Hardness (Core-SVP)						
BKZ block-size	283	439	681	907	1353	1816
Classical	82	128	199	264	395	531
Quantum	76	117	182	243	362	488
MSIS Hardness (Core-SVP)						
BKZ block-size (SUF)	384(274)	560(442)	811(658)	1183(880)	1587(1317)	2379(1802)
Classical (SUF)	112(80)	163(129)	236(192)	345(256)	463(384)	694(526)
Quantum (SUF)	101 (72)	148(117)	214(174)	313(233)	420(349)	630(477)
pk (bytes)	759	1264	1677	2528	3797	5824
sig (bytes)	1402	1440	3182	2976	7435	7474
pk+sig (bytes)	2161	2704	4859	5504	11232	13298

Under the parameter setting in Table 11, the efficiency of the BiT signature scheme is comparable to that of ML-DSA and is relatively efficient. We also provided the parameters and results for the short-sized version, as shown in Table 12.

Table 12: Parameter, Security, Size for Compactness (Short Size)

Security Level	128	256	512
η	1/16	3/64	1/16
q	3329	3329	7681
d	256	512	1024
(n, m)	(3, 3)	(3, 3)	(3,3)
τ	30	58	115
β	89	185	384
γ_1	1840	5600	16050
γ_2	480	380	1120
B_2	4283	8131	26772
B_{h_∞}	1	1	1
B_{h_2}	20	30	34
δ_κ	1.3	1.3	1.3
acceptance rate	10%	10%	10%
MLWE Hardness (Core-SVP)			
BKZ block-size	439	746	1335
Classical	128	263	512
Quantum	117	253	465
MSIS Hardness (Core-SVP)			
BKZ block-size (SUF)	516(452)	1097(894)	2192(1761)
Classical (SUF)	150(131)	320(261)	640(514)
Quantum (SUF)	136 (119)	290(236)	580(466)
pk (bytes)	1168	2336	5056
sig (bytes)	1104	2448	5520
pk+sig (bytes)	2272	4784	10576

Minimizing the combined size of the public key and signature: In many applications, such as certificate chains, the public key and signature are transmitted together. Therefore, the combined size of the two is a primary design objective. Our scheme focuses on minimizing combined size of the public key and signature.

Modular design: The scheme is constructed over module lattices using the polynomial ring $\mathbb{Z}_q[x]/\langle x^d + 1\rangle$ at all security levels. The main operations in signing and verification consist of

XOF expansion, SM3 hashing, and polynomial multiplication over $\mathbb{Z}_q[x]/\langle x^d + 1\rangle$. Supporting different security levels only requires adjusting the corresponding ring parameters and XOF settings. Consequently, optimized implementations can be easily adapted across different security levels.

References

[1] Martin R. Albrecht. Lattice Estimator. `https://github.com/malb/lattice-estimator`.

[2] Martin R. Albrecht, Benjamin R. Curtis, Amit Deo, Alex Davidson, Rachel Player, Eamonn W. Postlethwaite, Fernando Virdia, and Thomas Wunderer. Estimate all the {LWE, NTRU} schemes! In *Security and Cryptography for Networks - 11th International Conference, SCN 2018, Amalfi, Italy, September 5-7, 2018, Proceedings*, pages 351–367, 2018.

[3] Erdem Alkim, Léo Ducas, Thomas Pöppelmann, and Peter Schwabe. Newhope without reconciliation. *IACR Cryptology ePrint Archive*, 2016:1157, 2016.

[4] Anja Becker, Léo Ducas, Nicolas Gama, and Thijs Laarhoven. New directions in nearest neighbor searching with applications to lattice sieving. In *Proceedings of the Twenty-Seventh Annual ACM-SIAM Symposium on Discrete Algorithms*, pages 10–24, 2016.

[5] Alexandre Berzati, Andersson Calle Viera, Maya Chartouny, Steven Madec, Damien Vergnaud, and David Vigilant. Exploiting intermediate value leakage in dilithium: a template-based approach. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2023(4):188–210, 2023.

[6] Zhaohui Chen, Emre Karabulut, Aydin Aysu, Yuan Ma, and Jiwu Jing. An efficient non-profiled side-channel attack on the crystals-dilithium post-quantum signature. In *2021 IEEE 39th International Conference on Computer Design (ICCD)*, pages 583–590. IEEE, 2021.

[7] Jung Hee Cheon, Hyeongmin Choe, Julien Devevey, Tim Güneysu, Dongyeon Hong, Markus Krausz, Georg Land, Marc Möller, Damien Stehlé, and MinJune Yi. HAETAE: Shorter lattice-based fiat-shamir signatures. Cryptology ePrint Archive, Paper 2023/624, 2023.

[8] Léo Ducas, Alain Durmus, Tancrède Lepoint, and Vadim Lyubashevsky. Lattice signatures and bimodal Gaussians. pages 40–56, 2013.

[9] Yong Liu, Yuejun Liu, Yongbin Zhou, Yiwen Gao, Zehua Qiao, and Huaxin Wang. A novel power analysis attack against crystals-dilithium implementation. In *2024 IEEE European Test Symposium (ETS)*, pages 1–6. IEEE, 2024.

[10] Yuejun Liu, Yongbin Zhou, Shuo Sun, Tianyu Wang, Rui Zhang, and Jingdian Ming. On the security of lattice-based fiat-shamir signatures in the presence of randomness leakage. *IEEE Transactions on Information Forensics and Security*, 16:1868–1879, 2020.

[11] Vadim Lyubashevsky, Léo Ducas, Eike Kiltz, Tancrède Lepoint, Peter Schwabe, Gregor Seiler, Damien Stehlé, and Shi Bai. CRYSTALS-DILITHIUM. Technical report, National Institute of Standards and Technology, 2022. available at `https://csrc.nist.gov/Projects/post-quantum-cryptography/selected-algorithms-2022`.

[12] Tolun Tosun and Erkay Savas. Zero-value filtering for accelerating non-profiled side-channel attack on incomplete ntt-based implementations of lattice-based cryptography. *IEEE Transactions on Information Forensics and Security*, 19:3353–3365, 2024.

[13] Vincent Quentin Ulitzsch, Soundes Marzougui, Mehdi Tibouchi, and Jean-Pierre Seifert. Profiling side-channel attacks on dilithium: A small bit-fiddling leak breaks it all. In *International Conference on Selected Areas in Cryptography*, pages 3–32. Springer, 2022.

[14] Huaxin Wang, Yiwen Gao, Yuejun Liu, Qian Zhang, and Yongbin Zhou. In-depth correlation power analysis attacks on a hardware implementation of crystals-dilithium. *Cybersecurity*, 7(1):21, 2024.

[15] J. Woo, J. Kim, and Ga Hee Hong. Ntru+sign: compact ntru-based signatures using bimodal distributions. *Designs, Codes and Cryptography*, 94(2), 2026.

[16] Yuanyuan Zhou, Weijia Wang, Yiteng Sun, and Yu Yu. Rejected signatures challenges pose new challenges: Key recovery of crystals-dilithium via side-channel attacks. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2025(4):817–847, 2025.